Конспект для подготовки к государственному экзамену магистерской программы «Интеллектуальные системы»

15 мая 2026 г.

Содержание

1 Билет 1. Парадигмы программирования 3

2 Билет 2. Принципы ООП и обобщенное программирование 6

3 Билет 3. Средства современных функциональных языков на примере Haskell 10

4 Билет 4. Логическое программирование и базовые механизмы Prolog 15

5 Билет 5. Последовательная и параллельная сложность алгоритмов 17

6 Билет 6. Логическая модель представления знаний 19

7 Билет 7. Сетевая модель представления знаний 21

8 Билет 8. Онтологии в инженерии знаний 23

9 Билет 9. Лингвистические онтологии и тезаурусы 25

10 Билет 10. Продукционная модель представления знаний 27

11 Билет 11. Поиск решений в пространстве состояний 29

12 Билет 12. Слепой поиск, Hill Climbing, Монте-Карло и эволюционные алгоритмы 31

13 Билет 13. Поиск решений для игр с полной информацией 33

14 Билет 14. Байесовские сети 35

15 Билет 15. Деревья решений, CART и C4.5 37

16 Билет 16. Нейронные сети прямого распространения: MLP и RBF 39

17 Билет 17. Ассоциативные правила: Apriori и FP-tree 41

18 Билет 18. Обучение с учителем и классификация текстов 43

19 Билет 19. Обучение без учителя и кластеризация текстов 45

20 Билет 20. Уровни языка и анализ текста 47

21 Билет 21. Языковые модели, n-граммы, сглаживание и перплексия 49

22 Билет 22. Дистрибутивная семантика, Word2Vec и FastText 51

23 Билет 23. Современные нейросетевые языковые модели и BERT 53

24 Билет 24. Машинный перевод **55**

25 Билет 25. Извлечение информации и анализ тональности **57**

26 Билет 26. Процессная модель управления проектом **59**

27 Билет 27. Сетевое планирование проекта **61**

1 Билет 1. Парадигмы программирования

Короткий план ответа

1. Дать определение парадигмы программирования.
2. Показать, что парадигма задает модель вычислений, структуру программы и способ декомпозиции задачи.
3. Сравнить императивное, функциональное и логическое программирование.
4. Для каждой парадигмы назвать теоретическую основу, ключевые понятия, достоинства, ограничения и примеры языков.

Короткая версия для письменного ответа

Парадигма программирования — это совокупность идей, понятий и методов, задающих фундаментальный стиль написания программ. Она определяет, как программист мыслит о задаче, как описывает вычисление и как организует программу.

Парадигма задает:

- **модель вычислений**: последовательное изменение состояния, вычисление функций или логический вывод;
- **структуру программы**: процедуры, функции, классы, факты и правила;
- **основные строительные блоки**: переменные, объекты, функции, отношения;
- **способ декомпозиции задачи**: разбиение на шаги, преобразования данных или логические утверждения.

Парадигма не совпадает с языком программирования: многие современные языки являются мультипарадигменными. Например, Python поддерживает императивный, объектно-ориентированный и функциональный стиль.

Императивное программирование

Суть. Вычисление описывается как последовательность команд, которые изменяют состояние программы. Программа отвечает на вопрос: *как именно выполнить задачу.*

Теоретическая и машинная основа. Императивный стиль близок к машине Тьюринга и архитектуре фон Неймана: есть память, команды и пошаговое изменение содержимого памяти.

Ключевые признаки:

- **состояние**: значения переменных и структур данных в конкретный момент времени;
- **присваивание**: операция, явно изменяющая состояние, например `x = x + 1`;
- **управляющие конструкции**: последовательность команд, ветвления `if`, циклы `for`/`while`;
- **побочные эффекты**: изменение внешнего состояния, например глобальных переменных, файлов, консоли или базы данных.

Преимущества. Императивный стиль интуитивен для алгоритмов, близок к реальной архитектуре компьютеров и часто позволяет эффективно управлять ресурсами.

Недостатки. В больших программах трудно контролировать изменяемое состояние. Побочные эффекты усложняют тестирование, доказательство корректности и параллельное выполнение.

Примеры языков. C, Pascal, Fortran; также C++, Java, Python и JavaScript поддерживают императивный стиль.

Функциональное программирование

Суть. Вычисление рассматривается как вычисление значений выражений и применение функций. Программа ближе к описанию того, *что нужно получить*, а не пошагового изменения памяти.

Теоретическая основа. Функциональное программирование опирается на лямбда-исчисление Алонзо Черча, где вычисление задается через абстракцию функций и применение функции к аргументу.

Ключевые признаки:

- **чистые функции**: при одинаковых аргументах дают одинаковый результат и не имеют побочных эффектов;
- **ссылочная прозрачность**: вызов чистой функции можно заменить его результатом без изменения поведения программы;
- **неизменяемость данных**: вместо изменения существующего значения создается новое;
- **функции как объекты первого класса**: функции можно передавать как аргументы, возвращать из других функций и сохранять в переменных;
- **функции высшего порядка**: функции, работающие с другими функциями, например `map`, `filter`, `fold`;
- **рекурсия**: часто используется вместо циклов;
- **ленивые вычисления**: в некоторых языках выражение вычисляется только тогда, когда его значение действительно нужно.

Преимущества. Чистые функции легче тестировать, комбинировать и распараллеливать. Неизменяемость данных уменьшает число ошибок, связанных с состоянием.

Недостатки. Для задач с большим количеством изменяемого состояния функциональный стиль может казаться менее естественным. Ввод-вывод и другие эффекты требуют специальных механизмов, например монад в Haskell. При неаккуратной реализации возможны накладные расходы на создание новых структур данных.

Примеры языков. Haskell, Lisp, Scheme, Clojure, OCaml, F#; Scala и JavaScript являются мультипарадигменными языками с поддержкой функционального стиля.

Логическое программирование

Суть. Программа задается как совокупность фактов и правил, а выполнение программы понимается как поиск доказательства запроса. Программист описывает, какие утверждения истинны и какие отношения существуют, а система вывода ищет значения переменных, при которых цель выполняется.

Теоретическая основа. Логическое программирование основано на логике предикатов первого порядка. На практике, например в Prolog, часто используется подмножество в виде дизъюнктов Хорна. Основные механизмы вывода — унификация, резолюция и бэктрекинг.

Ключевые признаки:

- **факты**: безусловные утверждения, например `parent(john, mary).`;
- **правила**: условные утверждения, например `ancestor(X,Y) :- parent(X,Y).`;
- **запросы**: цели, которые система пытается доказать;
- **отношения вместо функций**: одно отношение можно использовать в разных направлениях;
- **унификация**: подбор подстановок для переменных;
- **бэктрекинг**: возврат к альтернативам при неудаче или при поиске следующего решения.

Пример идеи. Если заданы факты `parent(john, mary).` и правило `ancestor(X,Y) :- parent(X,Y).`, то запрос `ancestor(john, Who)` ищет такое значение `Who`, при котором утверждение доказывается. В данном случае система найдет `Who = mary`.

Преимущества. Логический стиль дает высокий уровень абстракции и хорошо подходит для задач искусственного интеллекта, экспертных систем, символьных вычислений, обработки естественного языка и задач поиска.

Недостатки. Эффективность зависит от порядка правил, фактов и стратегии поиска. Для численных расчетов и низкоуровневого управления ресурсами логический стиль обычно менее удобен.

Примеры языков. Prolog, Mercury.

Сравнение парадигм

Императивная парадигма описывает вычисление как изменение состояния командами. Основные элементы: переменные, присваивания, циклы, процедуры.

Функциональная парадигма описывает вычисление как применение функций к значениям. Основные элементы: выражения, чистые функции, неизменяемые данные, функции высшего порядка.

Логическая парадигма описывает вычисление как доказательство цели из фактов и правил. Основные элементы: предикаты, факты, правила, запросы, унификация.

Главное различие можно сформулировать так: императивная программа говорит компьютеру, *как* получить результат; функциональная программа описывает, *какое значение* нужно вычислить; логическая программа описывает, *какие утверждения истинны*, а результат получается через логический вывод.

Что важно проговорить на экзамене

- Парадигма — это не синтаксис языка, а способ мышления о вычислении и структуре программы.
- Один язык может поддерживать несколько парадигм.
- Императивная парадигма: вычисление как последовательное изменение состояния.
- Функциональная парадигма: вычисление как применение функций и преобразование значений.
- Логическая парадигма: вычисление как доказательство запроса на основе фактов и правил.
- Для билета 1 достаточно объяснить унификацию, резолюцию и бэктрекинг на уровне идеи; подробный механизм Prolog относится к билету 4.

2 Билет 2. Принципы ООП и обобщенное программирование

Короткий план ответа

1. Дать определение объектно-ориентированного программирования.
2. Раскрыть абстракцию, инкапсуляцию и полиморфизм.
3. Перечислить виды полиморфизма и отличить статический полиморфизм от динамического.
4. Объяснить концепцию обобщенного программирования и его связь с параметрическим полиморфизмом.

Короткая версия для письменного ответа

Объектно-ориентированное программирование — это парадигма, в которой программа представляется как совокупность взаимодействующих объектов. Объект имеет состояние, поведение и идентичность; обычно он является экземпляром класса. Класс описывает общие атрибуты и методы для группы объектов. Классы могут образовывать иерархии наследования, но ООП не сводится только к наследованию.

Главная идея ООП — моделировать предметную область через объекты и их взаимодействия. Это помогает управлять сложностью, локализовать изменения, повторно использовать код и строить расширяемые системы.

Абстракция

Абстракция — это выделение существенных свойств и поведения объекта при отвлечении от несущественных деталей реализации. Абстракция отвечает на вопрос: *что важно знать об объекте в данной задаче.*

Зачем нужна абстракция:

- уменьшает сложность системы;
- позволяет работать с объектом через понятный интерфейс;
- отделяет смысловую модель предметной области от технических деталей реализации.

Как реализуется:

- через классы, которые задают общие поля и методы объектов;
- через интерфейсы и абстрактные классы, которые описывают контракт: что объект должен уметь делать, не фиксируя конкретный способ реализации.

Пример. В библиотечной системе сущность `Book` можно описать через название, автора, ISBN и операции `issue()` и `returnBook()`. Детали хранения в базе данных или расположения книги на полке не входят в эту абстракцию, если они не важны для текущей задачи.

Инкапсуляция

Инкапсуляция — это объединение данных и методов, работающих с этими данными, внутри объекта, а также ограничение прямого доступа к внутреннему состоянию объекта.

Инкапсуляция тесно связана с **сокрытием данных**: объект предоставляет внешний интерфейс, а детали реализации оставляет внутри. Поэтому другие части программы зависят не от внутреннего устройства объекта, а от его публичного поведения.

Зачем нужна инкапсуляция:

- защищает состояние объекта от некорректного изменения;
- сохраняет инварианты класса;
- повышает модульность: реализацию можно менять, если публичный интерфейс остается прежним;
- упрощает сопровождение и тестирование.

Как реализуется:

- модификаторами доступа: `private`, `protected`, `public`;
- публичными методами, через которые объект контролирует чтение и изменение своего состояния;
- методами с проверками, а не прямым доступом к полям.

Пример. В классе банковского счета поле `balance` делают закрытым. Изменять баланс можно только через методы `deposit(amount)` и `withdraw(amount)`, где проверяется положительность суммы и достаточность средств.

Полиморфизм

Полиморфизм — это возможность использовать единый интерфейс для работы с объектами или значениями разных типов, при этом конкретное поведение определяется типом объекта или способом параметризации.

В ООП полиморфизм позволяет писать код, который зависит от общего интерфейса, а не от конкретных классов. Это уменьшает число проверок вида `if`/`switch` по типу объекта и облегчает расширение системы.

Виды полиморфизма

1. Полиморфизм подтипов также называют включающим полиморфизмом. Объект класса-потомка можно использовать там, где ожидается объект базового класса или интерфейса. Конкретная реализация метода выбирается во время выполнения через динамическую диспетчеризацию.

```
Shape[] shapes = { new Circle(), new Rectangle() };
for (Shape s : shapes) {
    s.draw(); // вызывается draw конкретной фигуры
}
```

Этот вид полиморфизма связан с наследованием, интерфейсами, переопределением методов и поздним связыванием.

2. Ad-hoc полиморфизм означает, что одно имя операции используется для разных типов, но реализации могут быть независимыми. Типичные формы:

- перегрузка функций или методов;
- перегрузка операторов.

Например, метод `print` может иметь версии для `int`, `double` и `String`. Обычно выбор версии происходит на этапе компиляции, поэтому это статический полиморфизм.

3. Параметрический полиморфизм позволяет писать функции и классы с параметрами типов. Один алгоритм работает для разных типов, если эти типы удовлетворяют нужным требованиям.

Примеры: `List<T>` в Java/C#, `vector<T>` и шаблонные функции в C++.

4. Полиморфизм приведения связан с автоматическими или явными преобразованиями типов. Например, значение типа `int` может быть приведено к `double`. Этот вид обычно менее важен для ООП, но его иногда выделяют в общей классификации полиморфизма.

Статический и динамический полиморфизм

Статический полиморфизм разрешается на этапе компиляции. К нему относят перегрузку и многие реализации обобщений/шаблонов.

Динамический полиморфизм разрешается во время выполнения. К нему относят подтипный полиморфизм с виртуальными методами.

Обобщенное программирование

Обобщенное программирование — это подход, при котором алгоритмы и структуры данных пишутся независимо от конкретных типов данных. Типы задаются как параметры, а один и тот же код можно использовать для разных типов.

Цели обобщенного программирования:

- повторное использование кода;
- типобезопасность: ошибки типов обнаруживаются как можно раньше;
- отделение алгоритма от конкретного представления данных;
- эффективность, особенно в языках, где обобщенный код специализируется компилятором.

Реализация в языках:

- **C++**: шаблоны `template`, стандартная библиотека STL, контейнеры `vector<T>`, `list<T>`, алгоритмы вроде `sort`;
- **Java**: generics, например `ArrayList<E>` и `HashMap<K,V>`; в Java используется стирание типов;
- **C#**: generics, например `List<T>` и `Dictionary<TKey,TValue>`; информация о типах сохраняется во время выполнения.

Обобщенное программирование практически реализует идею **параметрического полиморфизма**: одна функция или структура данных параметризуется типом. В отличие от подтипного полиморфизма, здесь не обязательно нужна иерархия наследования. В отличие от перегрузки, обычно пишется одна общая реализация, а не набор отдельных реализаций для каждого типа.

Сравнение ключевых понятий

Абстракция скрывает несущественные детали и выделяет важную модель объекта.

Инкапсуляция скрывает внутреннее состояние и защищает его через контролируемый интерфейс.

Полиморфизм позволяет работать с разными типами через общий интерфейс или общий код.

Обобщенное программирование позволяет писать алгоритмы и структуры данных, параметризованные типами.

Что важно проговорить на экзамене

- ООП организует программу вокруг объектов, их состояния, поведения и взаимодействий.
- Абстракция отвечает за выделение существенного, инкапсуляция — за защиту внутреннего состояния, полиморфизм — за единый интерфейс для разных реализаций.
- Подтипный полиморфизм обычно динамический: выбор метода происходит во время выполнения.
- Перегрузка и шаблоны/дженерики чаще относятся к статическому полиморфизму.

- Обобщенное программирование основано на параметрическом полиморфизме и позволяет не переписывать алгоритм под каждый тип.

3 Билет 3. Средства современных функциональных языков на примере Haskell

Короткий план ответа

1. Кратко охарактеризовать Haskell как чисто функциональный язык.
2. Объяснить чистые функции, ссылочную прозрачность и управление эффектами.
3. Рассказать о функциях высшего порядка, каррировании и частичном применении.
4. Описать статическую типизацию, вывод типов, алгебраические типы данных и классы типов.
5. Объяснить ленивые вычисления, сопоставление с образцом и монады.

Короткая версия для письменного ответа

Haskell — чисто функциональный язык программирования общего назначения со статической типизацией, выводом типов и ленивыми вычислениями. В Haskell программа строится как набор выражений и функций, а не как последовательность команд, изменяющих состояние.

Основные средства Haskell:

- чистые функции и ссылочная прозрачность;
- функции высшего порядка;
- каррирование и частичное применение;
- статическая типизация и вывод типов;
- параметрический полиморфизм;
- ленивые вычисления;
- сопоставление с образцом;
- алгебраические типы данных;
- классы типов;
- монады для описания вычислений с контекстом и эффектами.

Чистота и ссылочная прозрачность

Чистая функция при одинаковых аргументах всегда возвращает одинаковый результат и не имеет побочных эффектов: не изменяет глобальные переменные, не выполняет ввод-вывод напрямую, не меняет внешнее состояние.

Ссылочная прозрачность означает, что выражение можно заменить его значением без изменения поведения программы. Например, если `square 3` равно `9`, то в чистой программе это выражение можно заменить на `9`.

Преимущества чистоты:

- легче рассуждать о корректности программы;
- функции проще тестировать изолированно;
- компилятору проще выполнять оптимизации;
- отсутствие общего изменяемого состояния облегчает параллелизм.

Побочные эффекты в Haskell не исчезают, а явно отражаются в типах. Например, `IO String` означает действие ввода-вывода, которое при выполнении возвращает строку. Благодаря этому чистая часть программы отделяется от взаимодействия с внешним миром.

Функции высшего порядка

В Haskell функции являются объектами первого класса: их можно передавать как аргументы, возвращать из других функций и связывать с именами.

Функция высшего порядка — функция, которая принимает другую функцию как аргумент или возвращает функцию как результат.

```
map    :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]

map (+1) [1,2,3]        -- [2,3,4]
filter even [1,2,3,4]  -- [2,4]
```

Композиция функций позволяет собирать сложное преобразование из простых:

```
(not . even) 3  -- True
```

Такие средства делают код декларативным: программа описывает цепочку преобразований данных.

Каррирование и частичное применение

Каррирование — представление функции от нескольких аргументов как последовательности функций от одного аргумента. В Haskell все функции формально принимают один аргумент.

```
add :: Int -> Int -> Int
add x y = x + y
```

Тип `Int -> Int -> Int` читается как `Int -> (Int -> Int)`: функция получает первый `Int` и возвращает функцию, ожидающую второй `Int`. Поэтому `add 5 3` интерпретируется как `(add 5) 3`.

Частичное применение — создание новой функции путем фиксации части аргументов:

```
addFive :: Int -> Int
addFive = add 5

addFive 3  -- 8
```

Статическая типизация и вывод типов

Haskell имеет строгую статическую типизацию: типы выражений проверяются на этапе компиляции. Многие ошибки обнаруживаются до запуска программы.

Система типов Haskell основана на идеях Хиндли–Милнера и поддерживает **вывод типов**: компилятор часто сам выводит наиболее общий тип выражения.

```
identity :: a -> a
identity x = x

apply :: (a -> b) -> a -> b
apply f x = f x
```

Буквы `a` и `b` обозначают переменные типов. Это пример **параметрического полиморфизма**: функция работает для любых типов, если ее определение не требует конкретных операций над ними.

Явные аннотации типов часто не обязательны, но считаются хорошим стилем: они документируют намерение программиста и помогают найти ошибки.

Ленивые вычисления

Ленивые вычисления означают, что выражение вычисляется только тогда, когда его результат действительно нужен. Такой способ называют *call-by-need*.

Преимущества ленивости:

- можно работать с бесконечными структурами данных;
- промежуточные структуры не всегда вычисляются полностью;
- управляющие конструкции можно задавать как обычные функции.

```
ones :: [Int]
ones = 1 : ones

take 5 ones  -- [1,1,1,1,1]
```

Недостатки ленивости: иногда сложнее предсказать порядок вычислений, использование памяти и производительность.

Сопоставление с образцом

Сопоставление с образцом позволяет анализировать структуру данных и извлекать ее части прямо в определениях функций.

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)

length' :: [a] -> Int
length' []     = 0
length' (_:xs) = 1 + length' xs
```

Сопоставление с образцом делает код компактным и наглядным: разные формы входных данных описываются разными уравнениями функции.

Алгебраические типы данных

Алгебраические типы данных позволяют строить собственные типы из вариантов и наборов полей.

Тип-сумма задает выбор между несколькими конструкторами:

```
data Shape
    = Circle Float
    | Rectangle Float Float

area :: Shape -> Float
area (Circle r)      = pi * r * r
area (Rectangle w h) = w * h
```

Тип-произведение объединяет несколько значений в одну структуру:

```
data Point = Point Float Float

data Person = Person
    { name :: String
    , age  :: Int
    }
```

Важные стандартные ADT:

```
data Maybe a = Nothing | Just a
```

`Maybe a` используется, когда результат может отсутствовать: `Just x` означает наличие значения, `Nothing` — отсутствие значения.

Классы типов

Класс типов в Haskell — механизм ad-hoc полиморфизма. Он задает набор операций, которые должен поддерживать тип, чтобы быть экземпляром этого класса.

Классы типов похожи на интерфейсы по идее контракта, но они описывают поведение типов, а не объектов.

Примеры стандартных классов типов:

- `Eq`: сравнение на равенство, операции `==` и `/=`;
- `Ord`: упорядочивание;
- `Show`: преобразование значения в строку;
- `Read`: чтение значения из строки;
- `Num`: числовые операции;
- `Functor`, `Applicative`, `Monad`: работа с вычислениями в контексте.

Ограничения на типы записываются через `=>`:

```
areEqual :: Eq a => a -> a -> Bool
areEqual x y = x == y
```

Эта функция работает для любого типа `a`, если для него определено сравнение на равенство.

Монады

Монада — это класс типов, который задает способ последовательно связывать вычисления с некоторым контекстом: возможной ошибкой, отсутствием значения, списком вариантов, состоянием или вводом-выводом.

Главная идея монады: результат одного шага передается в следующий шаг, при этом контекст вычисления сохраняется и обрабатывается единообразно.

Основные операции:

- `return` или `pure`: помещает обычное значение в контекст;
- `>>=`: связывает монадическое значение со следующей функцией.

Примеры монад:

- `Maybe`: вычисление может не дать результата;
- `Either e`: вычисление может завершиться ошибкой;
- список `[]`: вычисление может иметь несколько результатов;

- `IO`: действия ввода-вывода;
- `State`: вычисления с состоянием.

```
main :: IO ()
main = do
    putStrLn "What is your name?"
    name <- getLine
    putStrLn ("Hello, " ++ name ++ "!")
```

`do`-нотация является удобной записью последовательности монадических действий. В случае `IO` она позволяет описывать ввод-вывод, не нарушая чистоту обычных функций: эффект явно виден в типе.

Что важно проговорить на экзамене

- Haskell — чисто функциональный язык: вычисление строится вокруг выражений и функций.
- Чистота означает отсутствие неявных побочных эффектов; эффекты отражаются в типах, например `IO a`.
- Функции высшего порядка, каррирование и частичное применение дают мощные средства абстракции.
- Статическая типизация и вывод типов позволяют писать общий, но типобезопасный код.
- ADT и pattern matching являются основными средствами моделирования и разбора данных.
- Type classes реализуют ad-hoc полиморфизм, а монады позволяют единообразно описывать вычисления с контекстом.

4 Билет 4. Логическое программирование и базовые механизмы Prolog

Короткий план ответа

1. Объяснить идею логического программирования: программа как теория, вычисление как доказательство.
2. Описать факты, правила, запросы и дизъюнкты Хорна.
3. Раскрыть унификацию, SLD-резолюцию, бэктрекинг и отсечение.
4. Сравнить декларативную и процедурную семантику логических программ.

Короткая версия для письменного ответа

Логическое программирование — парадигма, в которой программа задается как набор логических утверждений: фактов и правил. Выполнение программы понимается как попытка доказать запрос на основе этих утверждений.

Центральная идея: **программа есть теория**, а **вычисление есть конструктивное доказательство цели**. Если цель содержит переменные, результатом доказательства становятся найденные подстановки для этих переменных.

Программа, факты, правила и запросы

В Prolog программа состоит из клауз, обычно записанных в виде фактов и правил.

```
parent(john, mary).
ancestor(X, Y) :- parent(X, Y).

?- ancestor(john, Who).
```

- **Факт** утверждает безусловную истинность отношения.
- **Правило** имеет голову и тело: голова истинна, если истинны все подцели тела.
- **Запрос** задает цель, которую система должна доказать.
- **Дизъюнкты Хорна** — ограниченный вид формул логики первого порядка, удобный для автоматического вывода.

Если запрос доказан, Prolog возвращает `true` и значения переменных. Если доказательство не найдено, возвращается `false`.

Унификация

Унификация — сопоставление двух термов с целью сделать их синтаксически одинаковыми путем подстановки значений вместо переменных.

- Две одинаковые константы унифицируются.
- Переменная может быть связана с термом.
- Структуры унифицируются, если имеют одинаковый функтор, одинаковую арность и попарно унифицируемые аргументы.

Например, `f(A,b)` и `f(c,B)` унифицируются с подстановкой `A = c`, `B = b`. Наиболее общий унификатор называют MGU.

SLD-резолюция

SLD-резолюция — стратегия вывода, используемая Prolog для доказательства целей.

1. Берется исходный запрос.
2. Из списка целей выбирается подцель, обычно самая левая.
3. Prolog просматривает факты и правила сверху вниз и ищет голову, унифицируемую с выбранной подцелью.
4. Если найден факт, подцель удаляется.
5. Если найдено правило, подцель заменяется телом правила.
6. Найденная подстановка применяется к оставшимся целям.
7. Если список целей пуст, доказательство успешно.

Таким образом, логический вывод одновременно является и поиском доказательства, и вычислением значений переменных.

Бэктрекинг и отсечение

Бэктрекинг — поиск с возвратом. Если текущий путь вывода не приводит к успеху или пользователь запрашивает следующее решение, Prolog возвращается к последней точке выбора и пробует альтернативный факт или правило.

Порядок правил, фактов и подцелей влияет на эффективность и даже на завершимость программы.

Отсечение ! управляет бэктрекингом:

- **зеленое отсечение** используется как оптимизация и не меняет логический смысл программы;
- **красное отсечение** меняет множество возможных решений и делает программу менее декларативной.

Декларативная и процедурная семантика

Декларативная семантика отвечает на вопрос: *что означает программа.* Программа рассматривается как набор логических утверждений, а запрос — как вопрос о логическом следовании.

Процедурная семантика отвечает на вопрос: *как Prolog выполняет программу.* Она включает выбор целей слева направо, просмотр правил сверху вниз, унификацию, SLD-резолюцию, бэктрекинг и отсечение.

В чистом логическом программировании желательно, чтобы декларативный смысл и процедурное поведение совпадали. На практике они могут расходиться: например, неудачный порядок рекурсивных правил может привести к бесконечному поиску, хотя декларативно решение существует.

Что важно проговорить на экзамене

- В логическом программировании вычисление трактуется как доказательство цели.
- Базовые элементы Prolog: факты, правила, запросы, термы и переменные.
- Унификация подбирает значения переменных, SLD-резолюция строит доказательство, бэктрекинг перебирает альтернативы.
- Декларативная семантика описывает смысл программы, процедурная — порядок ее выполнения.
- Отсечение **!** полезно для управления поиском, но может разрушать декларативность.

5 Билет 5. Последовательная и параллельная сложность алгоритмов

Короткий план ответа

1. Определить последовательную сложность и асимптотические оценки.
2. Ввести параллельное время, работу, стоимость, ускорение и эффективность.
3. Объяснить информационный граф алгоритма и критический путь.
4. Раскрыть ресурс параллелизма и его связь с достижимым ускорением.

Короткая версия для письменного ответа

Последовательная сложность оценивает ресурсы алгоритма при выполнении на одном процессоре: время $T(n)$ и память $S(n)$ как функции размера входа n.

Параллельная сложность оценивает выполнение алгоритма на нескольких процессорах с учетом зависимостей, коммуникаций и синхронизации. Главные величины: работа $T_1(n)$, то есть общее число операций; параллельное время $T_p(n)$ на p процессорах; критический путь $T_\infty(n)$, то есть самая длинная цепочка зависимых операций.

Последовательная сложность

Для сравнения алгоритмов используют асимптотический анализ:

- $O(g(n))$ — верхняя граница роста;
- $\Omega(g(n))$ — нижняя граница;
- $\Theta(g(n))$ — точная асимптотическая граница.

Типичные классы: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, полиномиальные, экспоненциальные и факториальные. Полиномиальные алгоритмы обычно считаются практически более приемлемыми, чем экспоненциальные, хотя реальная применимость зависит от констант и размера входа.

Параллельная сложность

$T_p(n)$ время выполнения на p процессорах.

$T_1(n)$ работа, то есть суммарное число операций; это время на одном процессоре для данного алгоритма.

$C_p(n)$ стоимость: $C_p(n) = pT_p(n)$.

$S_p(n)$ ускорение: $S_p(n) = T_1(n)/T_p(n)$ или отношение лучшего последовательного времени к параллельному.

$E_p(n)$ эффективность: $E_p(n) = S_p(n)/p$.

Для любого расписания верны нижние оценки:

$$T_p(n) \geq \frac{T_1(n)}{p}, \qquad T_p(n) \geq T_\infty(n).$$

Первая оценка связана с общим объемом работы, вторая — с зависимостями, которые нельзя выполнить параллельно.

Идеальное ускорение равно p, но на практике оно ограничено последовательными участками, зависимостями данных, синхронизацией и обменами.

Закон Амдала. Если доля нераспараллеливаемой части равна f, то максимальное ускорение при бесконечном числе процессоров ограничено:

$$S_{\max} \leq \frac{1}{f}.$$

Информационный граф

Информационный граф алгоритма — ориентированный ациклический граф, где вершины соответствуют элементарным операциям, а дуги — информационным зависимостям. Дуга $u \to v$ означает, что операция v не может начаться до получения результата операции u.

Информационный граф позволяет определить:

- какие операции можно выполнять параллельно;
- общую работу T_1 как сумму весов вершин;
- критический путь T_∞ как самый длинный путь по зависимостям;
- нижнюю оценку параллельного времени даже при неограниченном числе процессоров.

Пример: в выражении $(a+b)(c+d)$ сложения $a+b$ и $c+d$ независимы и могут выполняться параллельно, а умножение зависит от обоих результатов.

Ресурс параллелизма

Ресурс параллелизма показывает среднее количество операций, которое потенциально можно выполнять одновременно:

$$\Pi(n) = \frac{T_1(n)}{T_\infty(n)}.$$

Он задает верхнюю границу ускорения:

$$S_p(n) \leq \min(p, \Pi(n)).$$

Если процессоров больше, чем ресурс параллелизма, часть процессоров будет простаивать.

Для суммирования n чисел деревом: $T_1(n) = O(n)$, $T_\infty(n) = O(\log n)$, поэтому $\Pi(n) = O(n/\log n)$.

Что важно проговорить на экзамене

- Последовательная сложность говорит о работе на одном процессоре, параллельная — о времени с учетом зависимостей.
- Работа T_1 и критический путь T_∞ — две главные характеристики параллельного алгоритма.
- Информационный граф показывает зависимости между операциями.
- Критический путь нельзя ускорить добавлением процессоров.
- Ресурс параллелизма ограничивает максимально полезное число процессоров.

6 Билет 6. Логическая модель представления знаний

Короткий план ответа

1. Дать определение логической модели представления знаний.
2. Описать применение логики предикатов первого порядка и этапы формализации.
3. Назвать основные трудности логического представления знаний.
4. Объяснить дескриптивные логики: концепты, роли, TBox/ABox и связь с логикой предикатов.

Короткая версия для письменного ответа

Логическая модель представления знаний описывает знания в виде формул некоторой формальной логики. Знания задаются как утверждения об объектах, их свойствах и отношениях, а новые знания получаются с помощью правил вывода.

Компоненты логической модели: формальный язык, семантика и механизм вывода.

Логика предикатов первого порядка

Логика предикатов первого порядка позволяет описывать:

- объекты через константы;
- переменные, пробегающие объекты предметной области;
- свойства и отношения через предикаты;
- функциональные зависимости через функциональные символы;
- общие и частные утверждения через кванторы $\forall$ и $\exists$.

Примеры:

$$\forall x(Student(x) \to Smart(x)),$$

$$\forall x \forall y \forall z((Parent(x, y) \land Parent(y, z)) \to Ancestor(x, z)).$$

Этапы представления знаний

1. Анализ предметной области: выделение объектов, свойств, отношений и типовых утверждений.
2. Выбор словаря: константы, предикаты, функции.
3. Формализация знаний: перевод естественно-языковых утверждений в логические формулы.
4. Построение базы знаний из фактов и общих правил.
5. Формулировка запросов и запуск вывода.

Основные сложности

- **Выразительность против вычислимости**: полная логика предикатов выразительна, но автоматический вывод в ней сложен и в общем случае неразрешим.
- **Формализация естественного языка**: неформальные знания часто контекстны и неоднозначны.
- **Монотонность**: в классической логике добавление фактов не отменяет старые выводы, что плохо подходит для рассуждений по умолчанию.
- **Неполнота и неопределенность**: классическая логика плохо выражает вероятностные и нечеткие знания без расширений.
- **Масштабируемость**: вывод на больших базах знаний может быть дорогим.

Дескриптивные логики

Дескриптивные логики — семейство разрешимых фрагментов логики предикатов, ориентированных на описание понятий предметной области и отношений между ними. Они лежат в основе OWL и многих онтологических систем.

Основные элементы:

- **индивиды** — конкретные объекты;
- **концепты** — классы индивидов, аналоги унарных предикатов;
- **роли** — бинарные отношения между индивидами.

Конструкторы концептов включают пересечение $C \sqcap D$, объединение $C \sqcup D$, отрицание $\neg C$, ограничения существования $\exists R.C$, всеобщности $\forall R.C$ и числовые ограничения.

TBox и ABox

TBox терминологическая часть онтологии: определения и включения концептов, например $Man \equiv Person \sqcap Male$ или $Student \sqsubseteq Person$.

ABox утверждения об индивидах, например $Person(John)$ или $hasChild(John, Mary)$.

Типовые задачи вывода: проверка выполнимости концепта, включения концептов, принадлежности индивида концепту и извлечение всех экземпляров концепта.

Связь с логикой предикатов

Дескриптивные логики переводятся в логику предикатов:

- концепт C соответствует унарному предикату $C(x)$;
- роль R соответствует бинарному предикату $R(x, y)$;
- $\exists R.C$ соответствует $\exists y(R(x, y) \wedge C(y))$;
- $C \sqsubseteq D$ соответствует $\forall x(C(x) \rightarrow D(x))$.

Главное преимущество дескриптивных логик — ограничение выразительности ради разрешимого и практически применимого вывода.

Что важно проговорить на экзамене

- Логические модели дают строгую семантику и формальный вывод.
- Логика предикатов выразительна, но сложна для автоматического вывода.
- Дескриптивные логики описывают концепты, роли и индивиды.
- TBox хранит терминологию, ABox — факты об экземплярах.
- ДЛ являются фрагментами логики предикатов с контролируемой выразительностью.

7 Билет 7. Сетевая модель представления знаний

Короткий план ответа

1. Описать семантические сети и их свойства.
2. Раскрыть фреймы, их структуру, виды и межфреймовые связи.
3. Объяснить значения по умолчанию и немонотонный вывод.
4. Дать понятие графа знаний и привести примеры.

Короткая версия для письменного ответа

Сетевая модель представления знаний описывает знания в виде графа: вершины соответствуют объектам, понятиям или событиям, а ребра — отношениям между ними. К сетевым моделям относят семантические сети, фреймовые системы и современные графы знаний.

Семантические сети

Семантическая сеть — ориентированный граф, где узлы обозначают понятия или объекты, а дуги с метками обозначают семантические отношения.

Типичные отношения:

- **IS-A**: экземпляр класса или подкласс класса;
- **PART-OF**: часть-целое;
- **HAS-A**: обладание или наличие свойства;
- причинные, временные, пространственные и агентивные отношения.

Главные свойства семантических сетей:

- наглядность;
- ассоциативность;
- наследование свойств по отношениям типа IS-A;
- гибкость расширения;
- вывод через поиск путей и наследование.

Ограничения: неоднозначность меток дуг, слабая формальная семантика, трудности с кванторами, отрицанием и сложными логическими выражениями.

Фреймы

Фрейм — структура для представления типичного объекта, понятия или ситуации через набор слотов и значений.

Фрейм обычно включает:

- имя фрейма;
- слоты, то есть атрибуты;
- значения слотов;
- фасеты слотов: `VALUE`, `DEFAULT`, `IF-NEEDED`, `IF-ADDED`, тип, диапазон, кардинальность.

Значением слота может быть простое значение, ссылка на другой фрейм или процедура-демон, вызываемая при обращении к слоту.

Виды и связи фреймов

- **Фреймы-прототипы** описывают класс объектов или типовую ситуацию.
- **Фреймы-экземпляры** описывают конкретный объект или случай.

Межфреймовые связи:
- **IS-A** или **AKO**: связь с более общим фреймом, наследование слотов;
- ссылки через значения слотов;
- связи потомков и экземпляров.

Значения по умолчанию и немонотонный вывод

Фреймы позволяют задавать значения по умолчанию. Если у экземпляра значение не указано, оно наследуется от прототипа.

Немонотонный вывод — вывод, который может быть отменен при добавлении новой информации. Пример: по умолчанию птицы летают; если известно только, что Твити — птица, делается вывод, что Твити летает. Если затем выясняется, что Твити — пингвин, вывод отменяется.

Такой вывод нужен для моделирования здравого смысла и рассуждений при неполной информации.

Графы знаний

Граф знаний — структурированное представление фактов о сущностях, их атрибутах и отношениях в виде графа. Часто факты задаются тройками: субъект, предикат, объект.

Пример тройки:

```
(Leonardo_da_Vinci, painted, Mona_Lisa)
(Mona_Lisa, located_in, Louvre)
```

Характерные черты графов знаний:
- сущности имеют идентификаторы;
- отношения типизированы;
- могут использоваться RDF, RDFS, OWL;
- поддерживаются интеграция данных, семантический поиск, рекомендации и вопросно-ответные системы.

Графы знаний можно рассматривать как современное развитие идей семантических сетей с большим масштабом и более формальными стандартами.

Что важно проговорить на экзамене

- Семантическая сеть представляет знания как граф понятий и отношений.
- Фрейм группирует знания об объекте или ситуации в слотах.
- Наследование значений по умолчанию является примером немонотонного вывода.
- Граф знаний хранит факты о сущностях в виде связанного графа, часто через тройки.
- Главное различие: семантические сети исторически более неформальны, графы знаний масштабнее и чаще стандартизованы.

8 Билет 8. Онтологии в инженерии знаний

Короткий план ответа

1. Дать понятие онтологии.
2. Описать состав онтологии и типы отношений концептов.
3. Перечислить классификации онтологий и привести примеры.
4. Назвать принципы и основные шаги построения онтологии.

Короткая версия для письменного ответа

Онтология в инженерии знаний — формальное описание предметной области, включающее концепты, отношения, свойства, аксиомы и экземпляры. Онтология задает общий словарь и правила интерпретации понятий, чтобы знания могли использоваться людьми и программными системами.

Состав онтологии

- **Концепты** или классы: основные понятия предметной области.
- **Экземпляры**: конкретные объекты, принадлежащие концептам.
- **Свойства данных**: атрибуты, связывающие объект с литеральным значением.
- **Объектные свойства**: отношения между экземплярами классов.
- **Аксиомы**: ограничения и утверждения, всегда истинные в рамках онтологии.
- **Правила и ограничения**: домены, диапазоны, кардинальность, транзитивность, симметричность.

Типы отношений концептов

- **IS-A** или **subClassOf**: отношение вид-род, например $Mammal \sqsubseteq Animal$.
- **instanceOf**: принадлежность индивида классу.
- **partOf/hasPart**: часть-целое.
- **Ассоциативные отношения**: причинность, лечение, использование, расположение и другие предметные связи.
- **Атрибутивные отношения**: связь объекта с характеристикой, например возрастом или датой.

Классификация онтологий

Онтологии классифицируют по разным основаниям:

- по степени формальности: неформальные, полуформальные, формальные;
- по глубине: легковесные без развитой системы аксиом и тяжеловесные с аксиомами;
- по охвату: верхнего уровня, предметные, прикладные;
- по методологии: лингвистические, строящиеся от слов к понятиям, и понятийные, строящиеся от концептов;
- по типу отношений: таксономии, партономии, атрибутивные, причинно-следственные, неоднородные.

Примеры

- **WordNet** — лексический ресурс и легковесная лингвистическая онтология.
- **Cyc** — формальная онтология общих знаний и здравого смысла.

- **SUMO** — онтология верхнего уровня.
- Доменные медицинские или биологические онтологии описывают специализированные предметные области.
- Рубрикаторы и классификаторы могут рассматриваться как прикладные легковесные онтологические ресурсы.

Принципы построения

- **Ясность**: определения должны быть понятны и недвусмысленны.
- **Непротиворечивость**: онтология не должна давать противоречивые выводы.
- **Расширяемость**: новые понятия должны добавляться без разрушения существующей структуры.
- **Минимальность**: не стоит вводить лишние сущности без необходимости.
- **Переиспользование**: полезно опираться на уже существующие онтологии и стандарты.

Основные шаги построения

1. Определить предметную область, цель и пользователей онтологии.
2. Сформулировать компетентностные вопросы, на которые онтология должна помогать отвечать.
3. Найти и оценить существующие онтологии для переиспользования.
4. Выделить ключевые термины и концепты.
5. Построить иерархию классов: сверху вниз, снизу вверх или комбинированно.
6. Определить свойства, отношения, домены, диапазоны и ограничения.
7. Создать экземпляры, если они входят в задачу.
8. Закодировать онтологию, например в RDF Schema или OWL.
9. Проверить полноту, непротиворечивость и соответствие целям.
10. Документировать и сопровождать онтологию.

Что важно проговорить на экзамене

- Онтология задает формальный словарь предметной области.
- В состав онтологии входят концепты, отношения, свойства, аксиомы и экземпляры.
- Важнейшие отношения: IS-A, instanceOf, partOf и предметные ассоциации.
- Верхнеуровневые онтологии описывают общие категории, доменные — конкретную область, прикладные — задачу.
- Построение онтологии начинается с цели и компетентностных вопросов.

9 Билет 9. Лингвистические онтологии и тезаурусы

Короткий план ответа

1. Определить тезаурус и лингвистическую онтологию.
2. Описать особенности их состава и построения.
3. Перечислить виды лингвистических онтологических ресурсов.
4. Раскрыть основные семантические отношения с примерами.

Короткая версия для письменного ответа

Тезаурус — словарь, в котором лексические единицы связаны семантическими отношениями: синонимией, антонимией, гипонимией, меронимией и ассоциациями. Он помогает навигации по смысловым связям слов.

Лингвистическая онтология — онтологический ресурс, описывающий язык или языковые значения: слова, значения, грамматические категории, семантические роли и отношения между ними.

Тезаурусы

Основные элементы тезауруса:

- лексические единицы: слова и устойчивые словосочетания;
- синсеты — группы слов с близким значением;
- краткие определения и примеры употребления;
- частеречная информация;
- семантические связи между словами или синсетами.

Примеры: WordNet, RuWordNet, RuThes, тезаурус Роже.

Виды тезаурусов:

- идеографические, где слова организованы по смысловым областям;
- информационно-поисковые, используемые для индексирования и поиска;
- WordNet-подобные, подробно описывающие лексическую систему языка.

Лингвистические онтологии

Лингвистическая онтология описывает не внешнюю предметную область, а языковую систему или связь языковых выражений с понятиями.

Она может включать:

- лексические единицы, морфемы и фразы;
- грамматические категории: часть речи, падеж, число, время;
- синтаксические конструкции;
- семантические роли: агент, пациенс, инструмент;
- правила согласования, словообразования и интерпретации.

Особенность построения: часто используется путь **от слов к понятиям**. Анализируются реальные тексты и словоупотребления, затем выделяются значения и концепты. Поэтому перенос такой онтологии на другой язык требует адаптации: слова и границы значений в разных языках не совпадают полностью.

Виды лингвистических ресурсов

- **Однородные классификации**: ресурсы узкой области с одним типом отношений.
- **Тезаурусы**: понятия или значения связаны синонимией, иерархией и ассоциациями.

- **Рубрикаторы**: иерархические или фасетные классификации понятий и терминов.
- **Формальные лингвистические онтологии**: ресурсы в RDF/OWL с классами, свойствами и ограничениями.

Семантические отношения

Синонимия близость значений: *автомобиль, машина, авто.*

Антонимия противоположность: *большой – маленький.*

Гипонимия/гиперонимия отношение вид-род: *пудель* — гипоним слова *собака*; *собака* — гипероним слова *пудель.*

Меронимия/холонимия отношение часть-целое: *колесо* — часть *автомобиля.*

Тропонимия для глаголов: более специфичный способ действия, например *шептать* как способ *говорить.*

Причинность и следование одно событие вызывает или влечет другое: *убить* влечет *умереть.*

Ассоциация смысловая или тематическая связь без строгого логического типа.

В формальных онтологиях отношения могут быть представлены через `rdfs:subClassOf`, `rdf:type`, объектные свойства, свойства данных, эквивалентность, несовместимость, транзитивность и симметричность.

Что важно проговорить на экзамене

- Тезаурус фокусируется на словах и значениях, онтология — на формальном описании концептов и отношений.
- Синсет объединяет лексические единицы с одним или близким значением.
- Лингвистические онтологии часто строятся от реального словоупотребления к понятиям.
- Главные отношения: синонимия, антонимия, гипонимия, меронимия, ассоциация.
- WordNet можно рассматривать как тезаурус и как легковесную лексическую онтологию.

10 Билет 10. Продукционная модель представления знаний

Короткий план ответа

1. Определить продукционную систему и структуру продукционного правила.
2. Описать базу правил, рабочую память и механизм вывода.
3. Раскрыть цикл работы продукционной системы и направления вывода.
4. Объяснить обработку нечетких знаний в продукционных системах.

Короткая версия для письменного ответа

Продукционная модель представляет знания в виде правил вида *если условие, то действие*. Продукционная система состоит из базы правил, рабочей памяти с текущими фактами и механизма вывода, который сопоставляет правила с фактами и применяет подходящие правила.

Структура продукционной системы

База правил содержит продукции вида `IF условие THEN действие`.

Рабочая память содержит текущие факты задачи: исходные данные и промежуточные выводы.

Механизм вывода управляет применением правил к фактам.

Левая часть правила называется **антецедентом**; она задает условия применимости. Правая часть называется **консеквентом**; она может добавлять, изменять или удалять факты, запускать внешние процедуры или завершать вывод.

Цикл работы

Механизм вывода работает по циклу **сопоставление** – **выбор** – **выполнение**.

1. **Сопоставление**: условия правил сравниваются с фактами в рабочей памяти.
2. **Формирование конфликтного множества**: находятся все правила, которые могут сработать.
3. **Разрешение конфликтов**: выбирается одно правило или одна его инстанциация.
4. **Выполнение**: правая часть выбранного правила изменяет рабочую память или выполняет действие.
5. Цикл повторяется до достижения цели, пустого конфликтного множества или команды остановки.

Стратегии разрешения конфликтов: приоритеты правил, специфичность, свежесть данных, порядок правил или эвристический выбор.

Прямой и обратный вывод

Прямой вывод идет от известных фактов к новым заключениям. Подходит для мониторинга, интерпретации данных и планирования.

Обратный вывод идет от цели к условиям, которые должны ее подтвердить. Подходит для диагностики и консультационных систем.

Достоинства и недостатки

Достоинства:

- правила просты и близки к экспертным формулировкам;
- базу знаний легко пополнять и изменять;
- возможно объяснение вывода через сработавшие правила;
- применимы прямой, обратный и смешанный вывод.

Недостатки:

- при большом числе правил вывод может быть медленным;
- трудно гарантировать полноту и непротиворечивость базы правил;
- управление конфликтами и метаправила усложняют систему;
- независимые правила могут неожиданно взаимодействовать.

Нечеткие знания

Классическая продукционная система работает с булевыми фактами. В реальных задачах часто встречаются нечеткие понятия: *высокая температура*, *молодой пациент*, *средний риск*.

Нечеткая продукционная система использует лингвистические переменные, нечеткие термы и функции принадлежности $\mu_A(x) \in [0, 1]$.

Пример правила:

```
IF temperature IS high AND pressure IS increased
THEN risk IS medium
```

Этапы нечеткого вывода:

1. **Фаззификация**: перевод четких входных значений в степени принадлежности термам.
2. **Агрегирование условий**: вычисление степени истинности условий, например через минимум для AND.
3. **Активация заключений**: перенос степени истинности условия на выходной терм.
4. **Аккумуляция**: объединение выводов нескольких правил.
5. **Дефаззификация**: получение четкого числового решения.

Нечеткие продукционные системы применяются в экспертных системах, управлении, поддержке принятия решений и распознавании образов.

Что важно проговорить на экзамене

- Продукционное правило имеет вид "если условие, то действие".
- Основные компоненты: база правил, рабочая память, механизм вывода.
- Цикл работы: сопоставление, конфликтное множество, выбор правила, выполнение.
- Прямой вывод идет от фактов к заключениям, обратный — от цели к подтверждающим фактам.
- Нечеткие продукционные системы работают со степенями принадлежности, а не только с истиной и ложью.

11 Билет 11. Поиск решений в пространстве состояний

Короткий план ответа

1. Описать задачу поиска в пространстве состояний.
2. Объяснить метод ветвей и границ.
3. Объяснить лучевой поиск.
4. Описать A*-алгоритм, условия оптимальности и частные случаи.

Короткая версия для письменного ответа

Пространство состояний — граф, где вершины соответствуют состояниям задачи, а дуги — действиям, переводящим одно состояние в другое. Цель поиска — найти путь от начального состояния к целевому, часто с минимальной стоимостью.

Основные компоненты: начальное состояние, операторы, целевой тест, стоимость пути и стратегия выбора следующего состояния для раскрытия.

Метод ветвей и границ

Branch and Bound — метод оптимизации, который строит дерево вариантов и отсекает ветви, заведомо не способные улучшить уже найденное решение.

Основные шаги:

1. **Ветвление**: задача разбивается на подзадачи.
2. **Оценка границ**: для каждой подзадачи вычисляется нижняя граница для минимизации или верхняя для максимизации.
3. **Отсечение**: если граница хуже текущего рекорда, ветвь не рассматривается.
4. **Обновление рекорда**: найденное допустимое решение сравнивается с лучшим известным.

Качество метода зависит от функции оценки границ и стратегии выбора узла: в глубину, в ширину или по наилучшей оценке. Применения: задача коммивояжера, целочисленное программирование, задача назначений.

Лучевой поиск

Beam Search — эвристический поиск, который на каждом уровне оставляет только B лучших кандидатов по оценочной функции. Число B называют шириной луча.

Алгоритм:

1. сгенерировать преемников текущих узлов;
2. оценить их эвристикой;
3. оставить только B лучших;
4. повторять до цели или исчерпания вариантов.

Лучевой поиск экономит память и время, но не является полным и не гарантирует оптимальность: хорошая ветвь может быть отброшена слишком рано. При $B = 1$ он близок к жадному поиску.

A*-алгоритм

A* — информированный алгоритм поиска кратчайшего пути, выбирающий узел с минимальной оценкой:

$$f(n) = g(n) + h(n),$$

где $g(n)$ — известная стоимость пути от старта до n, а $h(n)$ — эвристическая оценка стоимости от n до цели.

Шаги A*:

1. поместить стартовый узел в открытый список;
2. выбрать узел с минимальным $f(n)$;
3. если это цель, восстановить путь;
4. иначе раскрыть его преемников;
5. для каждого преемника обновить g, h, f и родителя, если найден лучший путь;
6. повторять, пока цель не найдена или открытый список не пуст.

Свойства A*

- **Допустимая эвристика**: $0 \leq h(n) \leq h^*(n)$, то есть эвристика не переоценивает реальную стоимость до цели.
- **Согласованная эвристика**: $h(n) \leq c(n, n') + h(n')$ для каждого перехода $n \to n'$.
- При допустимой эвристике A* оптимален для деревьев поиска; при согласованной — удобно работает на графах без повторного раскрытия закрытых узлов.
- Полнота выполняется при положительных стоимостях шагов и конечном ветвлении.

Частные случаи

Алгоритм Дейкстры получается при $h(n) = 0$: выбор идет только по уже накопленной стоимости $g(n)$.

Поиск в ширину получается при $h(n) = 0$ и единичных стоимостях всех шагов.

Жадный best-first search получается, если игнорировать $g(n)$ и выбирать по $h(n)$; он часто быстр, но не гарантирует оптимальность.

Что важно проговорить на экзамене

- Задача поиска задается начальными состоянием, операторами, целью и стоимостью.
- Branch and Bound ищет оптимум и отсекает ветви по оценкам границ.
- Beam Search ограничивает число рассматриваемых кандидатов и жертвует полнотой.
- A* балансирует уже пройденную стоимость g и прогноз h.
- Допустимость эвристики нужна для оптимальности A*.

12 Билет 12. Слепой поиск, Hill Climbing, Монте-Карло и эволюционные алгоритмы

Короткий план ответа

1. Определить слепой поиск и перечислить основные методы.
2. Описать Hill Climbing и его проблемы.
3. Объяснить метод Монте-Карло.
4. Описать эволюционные алгоритмы и их этапы.

Короткая версия для письменного ответа

Слепой поиск не использует эвристику близости к цели и опирается только на описание задачи: начальное состояние, операторы и целевой тест. К нему относят BFS, DFS, поиск с равномерной стоимостью, поиск с ограничением глубины и итеративное углубление.

Методы слепого поиска

BFS раскрывает состояния по уровням, использует очередь FIFO. Полон при конечном ветвлении и оптимален при одинаковой стоимости шагов, но требует много памяти: $O(b^d)$.

Uniform-Cost Search выбирает узел с минимальной стоимостью пути $g(n)$. Полон и оптимален при положительных стоимостях.

DFS идет по одной ветви в глубину, использует стек. Требует мало памяти $O(bm)$, но не полон при бесконечных путях и не оптимален.

Depth-Limited Search ограничивает глубину DFS числом L.

Iterative Deepening DFS многократно запускает ограниченный DFS с глубинами $0, 1, 2, \ldots$; сочетает полноту BFS и малую память DFS.

Hill Climbing

Восхождение к вершине — локальный жадный поиск, который переходит от текущего состояния к лучшему соседнему состоянию по целевой функции.

Шаги:

1. выбрать начальное состояние;
2. рассмотреть соседей;
3. перейти к соседу с лучшей оценкой;
4. остановиться, если все соседи не лучше текущего состояния.

Проблемы:

- локальные оптимумы;
- плато, где много состояний имеют одинаковую оценку;
- гребни, где нужен непрямой путь улучшения;
- зависимость от начальной точки.

Варианты: простое восхождение, наискорейшее восхождение, стохастическое восхождение, восхождение со случайными рестартами.

Метод Монте-Карло

Метод Монте-Карло — класс алгоритмов, использующих повторную случайную выборку для приближенного решения задачи.

В поиске и оптимизации он обычно работает так:

1. случайно генерируются возможные решения;
2. каждое решение оценивается целевой функцией;
3. выбирается лучшее или усредняется результат.

Методы Монте-Карло просты, применимы к большим и непрерывным пространствам, но дают вероятностный результат и часто сходятся медленно. В играх важен Monte Carlo Tree Search, где узлы оцениваются через случайные симуляции.

Эволюционные алгоритмы

Эволюционные алгоритмы — стохастические методы оптимизации, имитирующие естественный отбор. Они работают с популяцией возможных решений.

Общая схема:

1. создать начальную популяцию;
2. вычислить приспособленность каждого индивида;
3. выбрать родителей;
4. применить скрещивание и мутацию;
5. сформировать новое поколение;
6. повторять до критерия остановки.

Основные виды: генетические алгоритмы, эволюционные стратегии, эволюционное программирование, генетическое программирование.

Эволюционные алгоритмы полезны, когда функция сложная, негладкая, шумная, задана как черный ящик или пространство решений слишком велико для полного перебора. Их минусы — высокая вычислительная цена и чувствительность к параметрам: размеру популяции, вероятности мутаций, стратегии отбора.

Что важно проговорить на экзамене

- Слепой поиск не знает, насколько состояние близко к цели.
- BFS полон и оптимален при равных шагах, но дорог по памяти; DFS экономит память, но может зациклиться.
- Hill Climbing быстрый и простой, но застревает в локальных оптимумах.
- Монте-Карло использует случайные выборки и дает приближенные решения.
- Эволюционные алгоритмы ищут решение популяцией через отбор, скрещивание и мутацию.

13 Билет 13. Поиск решений для игр с полной информацией

Короткий план ответа

1. Определить игры с полной информацией и игровые деревья.
2. Объяснить минимаксный алгоритм.
3. Описать ограничение глубины и оценочную функцию.
4. Объяснить альфа-бета-отсечение и его эффективность.

Короткая версия для письменного ответа

В играх с полной информацией текущее состояние и допустимые ходы известны всем игрокам. Для двух игроков с нулевой суммой поиск хода часто сводится к анализу **игрового дерева**, где узлы — позиции, ребра — ходы, а листья — выигрыши, проигрыши или ничьи.

Игровые деревья

Компоненты игрового дерева:

- корень — текущая или начальная позиция;
- узлы — состояния игры;
- ребра — возможные ходы;
- листья — терминальные состояния;
- уровни обычно чередуют ходы MAX и MIN.

MAX стремится максимизировать оценку позиции, MIN — минимизировать ее.

Минимакс

Минимакс — рекурсивный алгоритм выбора оптимального хода при предположении, что оба игрока действуют рационально.

$$V(s) = \begin{cases} U(s), & \text{если } s \text{ терминальное состояние,} \\ \max\limits_{s' \in Succ(s)} V(s'), & \text{если ход MAX,} \\ \min\limits_{s' \in Succ(s)} V(s'), & \text{если ход MIN.} \end{cases}$$

Алгоритм:

1. построить дерево до терминальных узлов или заданной глубины;
2. оценить листья;
3. поднять оценки вверх: максимум на узлах MAX, минимум на узлах MIN;
4. выбрать ход из корня с лучшей оценкой.

Если дерево полностью конечно и противник оптимален, минимакс дает оптимальную стратегию. Сложность по времени $O(b^m)$, где b — фактор ветвления, m — глубина; память при DFS — $O(bm)$.

Ограничение глубины

Для сложных игр полное дерево недоступно. Поэтому поиск ограничивают глубиной d, а нетерминальные позиции на этой глубине оценивают **эвристической оценочной функцией**. Например, в шахматах простая оценка может быть разностью материальной ценности фигур.

Сила игрового агента сильно зависит от качества оценочной функции и порядка рассмотрения ходов.

Альфа-бета-отсечение

Альфа-бета-отсечение — оптимизация минимакса, которая не меняет результат, но не просматривает ветви, не способные повлиять на итоговое решение.

- α — лучшее значение, которое MAX уже может гарантировать.
- β — лучшее значение, которое MIN уже может гарантировать.

Отсечение происходит, когда дальнейший просмотр узла бесполезен:

- в MIN-узле можно отсекать, если текущее значение $v \leq \alpha$;
- в MAX-узле можно отсекать, если $v \geq \beta$.

В лучшем случае при хорошем упорядочивании ходов сложность снижается примерно с $O(b^m)$ до $O(b^{m/2})$, что позволяет просматривать дерево примерно вдвое глубже за то же время.

Что важно проговорить на экзамене

- Игровое дерево описывает все возможные продолжения игры.
- Минимакс предполагает оптимальную игру обоих игроков.
- MAX выбирает максимум, MIN выбирает минимум.
- При ограниченной глубине нужна эвристическая оценочная функция.
- Альфа-бета-отсечение ускоряет минимакс, не меняя выбранный ход.

14 Билет 14. Байесовские сети

Короткий план ответа

1. Дать определение байесовской сети.
2. Объяснить разложение совместного распределения и условную независимость.
3. Описать построение структуры и параметров сети.
4. Привести примеры использования.

Короткая версия для письменного ответа

Байесовская сеть — ориентированный ациклический граф, вершины которого соответствуют случайным переменным, а дуги отражают прямые вероятностные зависимости. Каждому узлу задается распределение $P(X_i \mid Parents(X_i))$.

Совместное распределение раскладывается как:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid Parents(X_i)).$$

Смысл модели

Байесовская сеть компактно представляет вероятностное пространство за счет условных независимостей. Если переменная имеет родителей, то при известных родителях она считается независимой от непотомков.

Количественная часть сети — таблицы условных вероятностей для дискретных переменных или параметры условных распределений для непрерывных.

Вероятностный вывод

Типичные запросы:

- вычислить апостериорную вероятность $P(Query \mid Evidence)$;
- найти наиболее вероятное объяснение наблюдений;
- предсказать ненаблюдаемые переменные по свидетельствам.

Алгоритмы вывода бывают точные, например исключение переменных и распространение доверия в деревьях, и приближенные, например MCMC и другие методы Монте-Карло.

Построение сети

Построение включает два аспекта.

Структура сети:

1. выбрать случайные переменные и их значения;
2. определить прямые зависимости между переменными;
3. построить ациклический ориентированный граф.

Структура может задаваться экспертно, обучаться из данных или строиться комбинированно.

Параметры сети:

- экспертные оценки вероятностей;
- оценка по полным данным через частоты и максимальное правдоподобие;
- обучение по неполным данным, например EM-алгоритмом;
- сглаживание при малом числе наблюдений.

Примеры

Медицинская диагностика. Переменные: сезон, аллергия, простуда, насморк, кашель, температура. По наблюдениям *насморк* = *да* и *кашель* = *да* можно вычислить вероятность простуды.

Фильтрация спама. Наивный байесовский классификатор является частным случаем байесовской сети, где класс письма влияет на признаки-слова, а признаки условно независимы при известном классе.

Другие применения: диагностика неисправностей, оценка рисков, биоинформатика, поддержка принятия решений.

Что важно проговорить на экзамене

- Байесовская сеть состоит из DAG и таблиц условных вероятностей.
- Граф кодирует условные независимости.
- Совместное распределение раскладывается в произведение локальных условных распределений.
- Структура и параметры могут задаваться экспертно или обучаться из данных.
- Наивный Байес — простой пример байесовской сети.

15 Билет 15. Деревья решений, CART и C4.5

Короткий план ответа

1. Описать деревья решений как модели классификации и регрессии.
2. Раскрыть критерии разбиения в CART и C4.5.
3. Объяснить ограничения роста дерева.
4. Описать отсечение и борьбу с переобучением.

Короткая версия для письменного ответа

Дерево решений — модель вида "если–то которая рекурсивно разбивает пространство признаков. Внутренние узлы задают проверки признаков, ветви — результаты проверок, листья — прогноз: класс, вероятности классов или числовое значение.

Деревья интерпретируемы и работают с разными типами признаков, но склонны к переобучению и нестабильны к небольшим изменениям данных.

Построение дерева

Дерево строится сверху вниз:

1. начать с корня, содержащего всю обучающую выборку;
2. выбрать лучший признак и порог/значение для разбиения;
3. разделить выборку на дочерние узлы;
4. рекурсивно повторять процесс;
5. остановиться по критерию остановки или затем выполнить pruning.

CART

CART строит бинарные деревья и применяется как для классификации, так и для регрессии.

Для классификации часто используется **индекс Джини**:

$$Gini(m) = 1 - \sum_{k=1}^{K} p_{mk}^2,$$

где p_{mk} — доля объектов класса k в узле m.

Лучшее разбиение максимизирует уменьшение нечистоты:

$$\Delta Gini = Gini(parent) - \sum_j \frac{N_j}{N} Gini(child_j).$$

Для регрессии CART минимизирует дисперсию или среднеквадратичную ошибку в листьях. Для числовых признаков перебираются возможные пороги, для категориальных — разбиения категорий на группы.

C4.5

C4.5 строит деревья классификации, допускает многозначные разбиения и умеет работать с пропущенными значениями.

Основной критерий связан с энтропией:

$$Entropy(S) = -\sum_{k=1}^{K} p_k \log_2 p_k.$$

Прирост информации:

$$Gain(S, A) = Entropy(S) - \sum_{v} \frac{|S_v|}{|S|} Entropy(S_v).$$

Обычный прирост информации склонен выбирать признаки с большим числом значений, поэтому C4.5 использует **gain ratio** — нормировку прироста информации на информацию разбиения.

Ограничение роста

Pre-pruning останавливает рост дерева заранее. Типичные параметры:

- максимальная глубина;
- минимальное число объектов в узле для разбиения;
- минимальное число объектов в листе;
- минимальное улучшение критерия качества;
- максимальное число листьев.

Ранняя остановка может быть слишком жадной и остановить дерево до полезных глубоких разбиений.

Отсечение

Post-pruning сначала строит большое дерево, затем удаляет ветви, которые ухудшают обобщение.

Основные подходы:

- reduced error pruning: проверка замены поддерева листом на валидационной выборке;
- pessimistic error pruning в C4.5: оценка ошибки с поправкой на сложность без отдельной валидации;
- cost-complexity pruning в CART: минимизация $R_\alpha(T) = R(T) + \alpha|L(T)|$, где $|L(T)|$ — число листьев.

Окончательное дерево часто выбирают по кросс-валидации.

Что важно проговорить на экзамене

- Дерево решений рекурсивно делит пространство признаков.
- CART строит бинарные деревья; для классификации часто использует Gini, для регрессии — MSE.
- C4.5 использует entropy, information gain и gain ratio.
- Главная проблема деревьев — переобучение.
- Ограничение роста и pruning уменьшают сложность дерева и улучшают обобщение.

16 Билет 16. Нейронные сети прямого распространения: MLP и RBF

Короткий план ответа

1. Определить сети прямого распространения.
2. Описать архитектуру MLP и функции активации.
3. Объяснить обучение через backpropagation.
4. Описать RBF-сети и их обучение.
5. Рассказать о переобучении и локальных минимумах.

Короткая версия для письменного ответа

Нейронные сети прямого распространения — сети, в которых информация движется от входного слоя к выходному без циклов и обратных связей. Они используются для классификации, регрессии и аппроксимации функций. Основные архитектуры в билете: MLP и RBF.

MLP

Многослойный перцептрон состоит из входного слоя, одного или нескольких скрытых слоев и выходного слоя. В полносвязном слое каждый нейрон получает выходы предыдущего слоя.

Нейрон вычисляет:

$$z = \sum_i w_i x_i + b, \qquad a = \varphi(z),$$

где w_i — веса, b — смещение, φ — функция активации.

Функции активации:

- sigmoid — часто для бинарной классификации, но страдает затухающими градиентами;
- tanh — центрирована около нуля, но тоже может насыщаться;
- ReLU — популярна в скрытых слоях, проста и уменьшает затухание градиента;
- Leaky ReLU — снижает проблему "мертвых"ReLU;
- linear — часто для регрессии;
- softmax — для многоклассовой классификации.

Без нелинейных функций активации многослойная сеть сводилась бы к линейной модели.

Обучение MLP

Обучение состоит в настройке весов и смещений для минимизации функции потерь.

Основной алгоритм — **обратное распространение ошибки**:

1. случайно инициализировать веса;
2. выполнить прямой проход и получить прогноз;
3. вычислить ошибку;
4. распространить градиенты от выхода к входу по правилу цепочки;
5. обновить веса, например $w \leftarrow w - \eta \frac{\partial L}{\partial w}$;
6. повторять по эпохам.

Популярные оптимизаторы: SGD, mini-batch SGD, RMSprop, Adam.

RBF-сети

RBF-сеть обычно имеет три слоя: входной, скрытый слой радиальных базисных функций и линейный выходной слой.

Скрытый нейрон имеет центр c_i и ширину σ_i. Часто используется гауссова функция:

$$\varphi_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma_i^2}\right).$$

Активация максимальна, когда вход близок к центру, поэтому RBF выполняет локальную аппроксимацию.

Обучение RBF часто двухэтапное:

1. определить центры, например алгоритмом k-means;
2. выбрать ширины;
3. обучить веса выходного линейного слоя как задачу линейной регрессии или градиентным методом.

MLP и RBF

MLP обычно выполняет глобальную аппроксимацию, обучается backpropagation, может быть глубоким.

RBF выполняет локальную аппроксимацию вокруг центров, часто обучается быстрее, но может требовать больше скрытых нейронов.

Переобучение

Переобучение возникает, когда сеть хорошо запоминает обучающую выборку, но плохо обобщает на новые данные. Причины: слишком сложная модель, мало данных, слишком долгое обучение, шум в данных.

Методы борьбы:

- L1/L2-регуляризация;
- dropout;
- ранняя остановка;
- аугментация данных;
- уменьшение сложности модели;
- валидационная выборка и кросс-валидация.

Локальные минимумы

Функция потерь нейронной сети невыпукла. Градиентные методы могут попадать в локальные минимумы или долго проходить седловые точки. На практике помогают удачная инициализация, mini-batch шум, оптимизаторы с моментом, Adam/RMSprop и несколько запусков обучения.

Что важно проговорить на экзамене

- Feedforward-сеть не имеет циклов: сигнал идет от входа к выходу.
- MLP использует линейные преобразования и нелинейные активации.
- Backpropagation вычисляет градиенты по правилу цепочки.
- RBF использует расстояния до центров и радиальные функции.
- Переобучение контролируют регуляризацией, dropout и ранней остановкой.

17 Билет 17. Ассоциативные правила: Apriori и FP-tree

Короткий план ответа

1. Определить ассоциативные правила, поддержку, достоверность и lift.
2. Описать два этапа задачи: поиск частых наборов и генерация правил.
3. Объяснить алгоритм Apriori.
4. Объяснить FP-tree и FP-Growth.

Короткая версия для письменного ответа

Обучение ассоциативным правилам ищет зависимости вида $X \Rightarrow Y$ в транзакционных данных. Классический пример: если покупатель купил хлеб и молоко, то он часто покупает масло.

Основные понятия

Itemset набор элементов.
Транзакция набор элементов, встретившихся вместе.
Правило выражение $X \Rightarrow Y$, где $X \cap Y = \varnothing$.

Поддержка показывает, как часто набор встречается:

$$supp(X \Rightarrow Y) = supp(X \cup Y) = \frac{|\{t \in T \mid X \cup Y \subseteq t\}|}{|T|}.$$

Достоверность показывает условную вероятность следствия при наличии посылки:

$$conf(X \Rightarrow Y) = \frac{supp(X \cup Y)}{supp(X)}.$$

Lift показывает, не объясняется ли правило простой популярностью следствия:

$$lift(X \Rightarrow Y) = \frac{conf(X \Rightarrow Y)}{supp(Y)}.$$

Если $lift > 1$, между X и Y есть положительная ассоциация.

Общая задача

1. Найти все **частые наборы**: itemsets с поддержкой не ниже `minSupport`.
2. Из частых наборов построить правила с достоверностью не ниже `minConfidence`.

Первый этап обычно самый дорогой вычислительно.

Apriori

Apriori основан на антимонотонности поддержки: если набор частый, то все его подмножества тоже частые; если набор нечастый, то все его надмножества нечастые.

Алгоритм:

1. посчитать поддержку всех 1-наборов и получить L_1;
2. из L_{k-1} сгенерировать кандидатов C_k;
3. удалить кандидатов, у которых есть нечастое $(k-1)$-подмножество;
4. просканировать базу и посчитать поддержку кандидатов;
5. оставить частые k-наборы L_k;

6. остановиться, когда L_k пусто.

После этого для каждого частого набора F перебираются непустые подмножества $X \subset F$ и проверяется правило $X \Rightarrow F \setminus X$.

Плюсы Apriori: простота и сильное отсечение кандидатов. Минусы: многократное сканирование базы и взрыв числа кандидатов при низкой поддержке.

FP-tree и FP-Growth

FP-Growth избегает явной генерации кандидатов. Он сжимает транзакционную базу в FP-дерево.

Построение:

1. первым проходом посчитать частоты элементов и оставить частые;
2. отсортировать частые элементы по убыванию поддержки;
3. вторым проходом вставить очищенные и отсортированные транзакции в FP-tree;
4. общие префиксы транзакций разделяют одни и те же пути, поэтому дерево компактно;
5. заголовочная таблица связывает узлы с одинаковыми элементами.

Поиск частых наборов:

- для каждого элемента строится условная база шаблонов;
- по ней строится условное FP-дерево;
- алгоритм рекурсивно извлекает частые наборы;
- если дерево содержит один путь, все комбинации элементов пути являются частыми.

FP-Growth обычно быстрее Apriori на плотных данных и при низком пороге поддержки, но сложнее в реализации и может требовать много памяти.

Что важно проговорить на экзамене

- Ассоциативное правило имеет вид $X \Rightarrow Y$.
- Support отвечает за частоту, confidence — за надежность правила, lift — за неслучайность связи.
- Apriori использует антимонотонность поддержки.
- FP-Growth строит FP-tree и не генерирует кандидатов явно.
- Правила строятся только из частых наборов и затем фильтруются по confidence.

18 Билет 18. Обучение с учителем и классификация текстов

Короткий план ответа

1. Дать определение обучения с учителем.
2. Описать задачу классификации.
3. Перечислить способы оценки качества классификации.
4. Объяснить признаки текстов и два метода классификации текстов.

Короткая версия для письменного ответа

Обучение с учителем — машинное обучение на размеченной выборке пар (x_i, y_i), где x_i — объект или его признаки, а y_i — правильный ответ. Модель учится по обучающим данным предсказывать ответы для новых объектов.

Основные задачи: классификация, где ответ — метка класса, и регрессия, где ответ — число.

Задача классификации

Классификация — построение алгоритма $a : X \to Y$, который относит объект к одному из заранее заданных классов $Y = \{c_1, \ldots, c_K\}$.

Типы:

- бинарная классификация: два класса;
- многоклассовая классификация: более двух классов;
- многометочная классификация: объект может иметь несколько меток.

Процесс обучения: сбор данных, выделение признаков, обучение модели, проверка на тестовой выборке, применение к новым объектам.

Оценка качества

Для бинарной классификации используют:

- TP — истинно положительные;
- TN — истинно отрицательные;
- FP — ложноположительные;
- FN — ложноотрицательные.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}.$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

Accuracy плохо отражает качество при дисбалансе классов. В многоклассовых задачах метрики усредняют по классам: macro-average дает равный вес каждому классу, micro-average считает суммарные TP, FP, FN.

Классификация текстов

Классификация текстов — автоматическое отнесение документа к категории: спам/не спам, тема новости, тональность отзыва.

Текст нужно преобразовать в числовые признаки:

- **Bag of Words**: учитывается наличие или частота слов, порядок игнорируется;
- **TF-IDF**: повышает вес слов, частых в документе, но редких в коллекции;
- **n-граммы**: учитывают локальный контекст;
- **лемматизация и стемминг**: уменьшают число разных форм слова;
- **стоп-слова**: часто удаляются как малоинформативные;
- **эмбеддинги**: плотные векторы слов или документов.

Наивный байесовский классификатор

Идея: выбрать класс, максимизирующий апостериорную вероятность:

$$a(x) = \arg\max_{y \in Y} P(y)P(x \mid y).$$

В текстах используется наивное предположение: признаки-слова условно независимы при известном классе:

$$P(x \mid y) \approx \prod_j P(x_j \mid y).$$

Для мультиномиальной модели вероятности слов оцениваются по частотам в документах класса. Обязательно используют сглаживание, чтобы неизвестные слова не давали нулевую вероятность.

Плюсы: простота, скорость, хорошая базовая модель для текстов. Минусы: предположение независимости слов грубое, взаимодействия между словами не учитываются.

Логистическая регрессия

Логистическая регрессия — линейный вероятностный классификатор. Для бинарной задачи:

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}.$$

Параметры обучаются минимизацией кросс-энтропии, часто с L1- или L2-регуляризацией. Для многоклассовой классификации используется softmax-регрессия.

Плюсы: дает вероятности, интерпретируема по весам признаков, хорошо работает с TF-IDF. Минусы: линейная граница решений, поэтому сложные нелинейные зависимости требуют дополнительных признаков или других моделей.

Что важно проговорить на экзамене

- Обучение с учителем требует размеченных примеров.
- Классификация предсказывает дискретную метку класса.
- Precision и recall важны при дисбалансе и разной цене ошибок.
- Для текстов ключевой этап — превращение текста в признаки.
- Наивный Байес быстр и прост; логистическая регрессия сильная линейная базовая модель.

19 Билет 19. Обучение без учителя и кластеризация текстов

Короткий план ответа

1. Дать определение обучения без учителя.
2. Описать задачу кластеризации и метрики качества.
3. Объяснить метод k-средних.
4. Объяснить иерархическую кластеризацию.
5. Рассказать о кластеризации текстов и признаках.

Короткая версия для письменного ответа

Обучение без учителя работает с неразмеченными данными и ищет скрытую структуру: кластеры, факторы, аномалии, ассоциации или представления меньшей размерности.

Кластеризация — разбиение объектов на группы так, чтобы объекты внутри кластера были похожи, а объекты из разных кластеров различались.

Оценка качества кластеризации

Оценка сложна, потому что правильных меток обычно нет.

Внутренние меры используют только данные:

- silhouette score: близко к 1 — объект хорошо лежит в своем кластере, около 0 — на границе, меньше 0 — вероятно не в своем кластере;
- индекс Данна: отношение межкластерного расстояния к внутрикластерному;
- индекс Дэвиса–Болдина: меньше — лучше.

Внешние меры используют известные метки, если они есть: Rand Index, Adjusted Rand Index, purity, F-мера.

Метод k-средних

k-means разбивает объекты на k кластеров, минимизируя сумму квадратов расстояний до центроидов.

Алгоритм:

1. выбрать k начальных центроидов;
2. отнести каждый объект к ближайшему центроиду;
3. пересчитать центроид как среднее объектов кластера;
4. повторять до стабилизации или лимита итераций.

Особенности:

- число кластеров k задается заранее;
- результат зависит от начальной инициализации;
- алгоритм сходится к локальному минимуму;
- хорошо работает для компактных примерно сферических кластеров;
- плохо работает с кластерами сложной формы, разной плотности и выбросами.

Выбор k: метод локтя, silhouette score, экспертная интерпретация.

Иерархическая кластеризация

Иерархическая кластеризация строит дерево кластеров — дендрограмму.

Виды:

- **агломеративная**: начинается с отдельных объектов и последовательно объединяет ближайшие кластеры;
- **дивизивная**: начинается с одного кластера и последовательно делит его.

Критерии расстояния между кластерами:

- single linkage: расстояние между ближайшими объектами, может давать эффект цепочки;
- complete linkage: расстояние между самыми дальними объектами, дает более компактные кластеры;
- average linkage: среднее расстояние между объектами;
- метод Уорда: минимизирует рост внутрикластерной суммы квадратов.

Плюс: не нужно заранее фиксировать число кластеров. Минус: высокая вычислительная сложность и необратимость ранних объединений.

Кластеризация текстов

Перед кластеризацией тексты преобразуют в векторы.

Этапы:

1. токенизация;
2. приведение к нижнему регистру;
3. удаление стоп-слов;
4. лемматизация или стемминг;
5. построение признаков: BoW, TF-IDF, n-граммы, эмбеддинги документов.

Для разреженных TF-IDF-векторов часто используют **косинусную близость**, потому что важнее направление вектора, а не его длина. Для плотных эмбеддингов могут использоваться косинусная или евклидова метрика.

Что важно проговорить на экзамене

- В обучении без учителя нет заранее заданных правильных ответов.
- Кластеризация ищет группы похожих объектов.
- k-means быстрый, но требует выбрать k и чувствителен к инициализации.
- Иерархическая кластеризация дает дендрограмму и позволяет выбрать число кластеров после построения.
- Для текстов особенно важны векторизация и метрика близости.

20 Билет 20. Уровни языка и анализ текста

Короткий план ответа

1. Перечислить уровни естественного языка.
2. Описать этапы многоуровневого анализа текста.
3. Сравнить инженерный подход и подход на машинном обучении.
4. Перечислить лингвистические ресурсы и их назначение.

Короткая версия для письменного ответа

Обработка естественного языка обычно строится как многоуровневый анализ: от символов и слов к морфологии, синтаксису, семантике, дискурсу и прагматике. Каждый уровень добавляет информацию, которая может использоваться следующими модулями.

Уровни естественного языка

- **Фонетика и фонология**: звуки речи и их функции; важны для распознавания и синтеза речи.
- **Морфология**: структура слов, словоформы, леммы, грамматические признаки.
- **Лексика и лексическая семантика**: значения слов, синонимия, омонимия, многозначность.
- **Синтаксис**: структура словосочетаний и предложений.
- **Семантика**: смысл фраз, предложений и текста.
- **Прагматика**: влияние контекста, целей и ситуации общения.
- **Дискурс**: связность текста выше уровня предложения.

Модули анализа текста

Типичный NLP-конвейер:

1. сегментация текста на предложения;
2. токенизация;
3. нормализация, лемматизация или стемминг;
4. POS-tagging и морфологический анализ;
5. синтаксический анализ: зависимости или составляющие;
6. распознавание именованных сущностей;
7. разрешение многозначности;
8. извлечение отношений и семантических ролей;
9. кореференция, дискурс-анализ, прагматическая интерпретация.

На практике порядок модулей зависит от задачи, а в нейросетевых системах несколько уровней могут обрабатываться одной моделью.

Инженерный подход

Подход на правилах использует вручную заданные грамматики, регулярные выражения, словари и шаблоны.

Плюсы:

- интерпретируемость;
- высокая точность в узких областях;
- не требует больших размеченных данных.

Минусы:

- трудоемкость разработки;
- хрупкость к вариативности языка;
- плохая переносимость на новые домены.

Подход на машинном обучении

ML-подход обучает модели на корпусах. Он может использовать обучение с учителем, без учителя, полуобучение и глубокие нейросетевые модели.

Плюсы:

- лучше масштабируется на большие данные;
- улавливает сложные статистические закономерности;
- часто дает более высокое качество на стандартных задачах.

Минусы:

- нужны данные и разметка;
- сложные модели менее интерпретируемы;
- качество сильно зависит от корпуса и домена.

Лингвистические ресурсы

Корпусы большие коллекции текстов: неразмеченные, POS-размеченные, синтаксические treebanks, семантические, NER-корпусы, параллельные корпусы.

Лексические ресурсы словари, тезаурусы, WordNet-подобные ресурсы, онтологии, списки именованных сущностей.

Грамматики и правила формальные грамматики, морфологические правила, шаблоны извлечения.

Ресурсы нужны для обучения и оценки моделей, лингвистического анализа, информационного поиска, машинного перевода и извлечения знаний.

Что важно проговорить на экзамене

- NLP анализирует текст на нескольких уровнях: морфология, синтаксис, семантика, дискурс.
- Конвейер обработки текста обычно начинается с токенизации и нормализации.
- Правила понятны и точны в узких задачах, но плохо масштабируются.
- ML-модели требуют данных, но лучше обобщают при хорошем обучении.
- Корпусы, словари, тезаурусы, онтологии и грамматики являются ключевыми лингвистическими ресурсами.

21 Билет 21. Языковые модели, n-граммы, сглаживание и перплексия

Короткий план ответа

1. Дать понятие языковой модели и перечислить виды.
2. Объяснить униграммные и n-граммные статистические модели.
3. Описать проблему разреженности и сглаживание.
4. Объяснить перплексию и способ ее подсчета.

Короткая версия для письменного ответа

Языковая модель задает вероятностное распределение на последовательностях слов и позволяет оценивать $P(w_1, \ldots, w_m)$ или вероятность следующего слова по контексту.

По цепному правилу:

$$P(w_1, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_1, \ldots, w_{i-1}).$$

Виды языковых моделей

- **Статистические**: основаны на частотах слов и n-грамм в корпусах.
- **Нейросетевые**: используют нейронные сети, эмбеддинги и контекстные представления.
- **Авторегрессионные**: предсказывают следующее слово по предыдущим.
- **Маскированные**: предсказывают скрытые токены по левому и правому контексту.

N-граммные модели

N-грамма — последовательность из N слов. N-граммная модель делает марковское предположение: вероятность слова зависит только от предыдущих $N-1$ слов:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-N+1}, \ldots, w_{i-1}).$$

Оценка максимального правдоподобия:

$$P(w_i \mid context) = \frac{Count(context, w_i)}{Count(context)}.$$

Униграммная модель игнорирует контекст: $P(W) = \prod_i P(w_i)$.
Биграммная модель учитывает одно предыдущее слово.
Триграммная модель учитывает два предыдущих слова.

Чем больше N, тем больше контекст, но тем сильнее проблема разреженности: многие допустимые n-граммы не встречаются в обучающем корпусе.

Применение

N-граммные модели применяются в распознавании речи, машинном переводе, автодополнении, проверке орфографии, информационном поиске и как признаки в классификации текстов.

Сглаживание

Сглаживание перераспределяет вероятностную массу от частых n-грамм к редким и невстречавшимся, чтобы избежать нулевых вероятностей.

Методы:

- **Лапласовское сглаживание**: добавить 1 к счетчикам;
- **add-δ**: добавить малое число δ;
- **интерполяция**: смешать модели разных порядков;
- **backoff**: при ненадежной n-грамме перейти к модели меньшего порядка.

Перплексия

Перплексия — внутренняя метрика качества языковой модели на тестовом корпусе. Чем ниже перплексия, тем лучше модель предсказывает текст.

Для последовательности $W = (w_1, \ldots, w_M)$:

$$PP(W) = P(W)^{-1/M}.$$

В логарифмической форме:

$$PP(W) = 2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i|w_1,\ldots,w_{i-1})}.$$

Интуитивно перплексия — среднее число равновероятных вариантов следующего слова, между которыми модель как будто выбирает. Сравнивать перплексию корректно только на одинаковых тестовых данных и словаре.

Что важно проговорить на экзамене

- Языковая модель оценивает вероятность текста или следующего слова.
- N-граммы ограничивают контекст последними $N - 1$ словами.
- Главная проблема n-грамм — разреженность и нулевые вероятности.
- Сглаживание делает вероятности ненулевыми и более устойчивыми.
- Перплексия ниже — модель лучше предсказывает тестовый текст.

22 Билет 22. Дистрибутивная семантика, Word2Vec и FastText

Короткий план ответа

1. Объяснить дистрибутивную гипотезу и понятие эмбеддинга.
2. Описать Word2Vec: CBOW, Skip-gram и способы обучения.
3. Описать FastText и учет подслов.
4. Раскрыть понятие семантического пространства слов.

Короткая версия для письменного ответа

Дистрибутивная семантика основана на гипотезе: слова, встречающиеся в похожих контекстах, имеют похожие значения. Поэтому значение слова можно представить числовым вектором, обученным на контекстах его употребления.

Эмбеддинг — плотный вектор слова в многомерном семантическом пространстве.

Свойства эмбеддингов

- близкие по смыслу слова имеют близкие векторы;
- векторы можно использовать как признаки в NLP-задачах;
- некоторые отношения отражаются как направления в пространстве;
- близость часто измеряют косинусной мерой.

Классический пример аналогии: $king - man + woman \approx queen$.

Word2Vec

Word2Vec — семейство нейросетевых моделей для обучения статических эмбеддингов слов. Основные архитектуры: CBOW и Skip-gram.

CBOW предсказывает центральное слово по окружающим словам. Контекстные слова переводятся в эмбеддинги, усредняются, затем модель предсказывает слово. CBOW обычно быстрее и хорошо работает с частыми словами.

Skip-gram предсказывает контекстные слова по центральному слову. Он часто лучше работает с редкими словами, но обучается медленнее.

Для ускорения обучения вместо полного softmax применяются:

- **hierarchical softmax**: слово кодируется путем в дереве;
- **negative sampling**: модель отличает реальные пары слово-контекст от случайных отрицательных пар.

FastText

FastText расширяет Word2Vec: слово представляется как набор символьных n-грамм. Вектор слова получается из векторов его подсловных частей.

Преимущества:

- лучше работает с редкими словами;
- может строить векторы для слов вне словаря;
- учитывает морфологию, что важно для языков с богатым словоизменением;
- полезен для ошибок, новых слов и производных форм.

FastText может обучаться в вариантах CBOW или Skip-gram, но вместо только целых слов использует также подсловные n-граммы.

Семантическое пространство

Обученные векторы образуют семантическое пространство, где:

- близость точек соответствует сходству значений;
- направления могут соответствовать отношениям: род, число, пол, страна-столица;
- пространство можно визуализировать через PCA или t-SNE.

Word2Vec и FastText дают **статические** эмбеддинги: у слова один вектор независимо от контекста. Современные модели вроде BERT строят **контекстуальные** эмбеддинги, где вектор слова зависит от предложения.

Что важно проговорить на экзамене

- Дистрибутивная гипотеза: значение слова определяется контекстами.
- Эмбеддинг — плотный числовой вектор слова.
- CBOW предсказывает слово по контексту, Skip-gram — контекст по слову.
- FastText использует символьные n-граммы и лучше справляется с редкими/OOV словами.
- Статические эмбеддинги не различают значения многозначного слова по контексту.

23 Билет 23. Современные нейросетевые языковые модели и BERT

Короткий план ответа

1. Описать особенности современных нейросетевых языковых моделей.
2. Раскрыть архитектуру Transformer.
3. Объяснить предобучение, transfer learning и fine-tuning.
4. Показать тонкую настройку на примере BERT.

Короткая версия для письменного ответа

Современные нейросетевые языковые модели используют большие корпуса и архитектуру Transformer, чтобы строить контекстуальные представления текста. В отличие от n-грамм, они способны учитывать дальние зависимости и переносить знания на разные NLP-задачи.

Transformer

Ключевые компоненты:

- **self-attention**: каждое слово учитывает другие слова последовательности;
- **multi-head attention**: несколько голов внимания выделяют разные типы зависимостей;
- **позиционные кодировки**: добавляют информацию о порядке слов;
- **feed-forward слои**: нелинейная обработка представлений;
- **residual connections и layer normalization**: стабилизируют обучение глубоких моделей.

Типы архитектур:

- encoder-only: BERT, RoBERTa; задачи понимания текста;
- decoder-only: GPT; генерация текста;
- encoder-decoder: T5, BART, машинный перевод и суммаризация.

Обучение

Обычно используется два этапа.

Предобучение на больших неразмеченных корпусах с задачами самообучения:

- Masked Language Modeling: предсказать замаскированные токены;
- Causal Language Modeling: предсказать следующий токен по предыдущим;
- дополнительные задачи, например NSP в исходном BERT.

Дообучение адаптирует предобученную модель к конкретной задаче на размеченных данных.

Transfer Learning

Transfer learning — перенос знаний, полученных на одной задаче, на другую связанную задачу. В NLP модель сначала учит общие закономерности языка на больших корпусах, а затем эти веса используются как начальная точка для классификации, NER, QA, анализа тональности и других задач.

Преимущества:

- меньше потребность в размеченных данных;
- выше качество на целевых задачах;

- быстрее обучение по сравнению с обучением с нуля.

BERT и fine-tuning

BERT — encoder-only модель Transformer, строящая двунаправленные контекстные представления. Она предобучалась на Masked Language Modeling и Next Sentence Prediction.

Тонкая настройка BERT:

1. выбрать предобученную модель;
2. подготовить вход: `[CLS]`, токены, `[SEP]`, attention mask;
3. добавить задачно-специфичную голову;
4. дообучить модель на размеченных данных с малой скоростью обучения;
5. оценить качество на тестовой выборке.

Примеры:

- классификация текста: использовать представление `[CLS]` и softmax-слой;
- NER: классифицировать каждый токен;
- QA: предсказать начало и конец ответа в тексте.

Варианты: полное дообучение всех параметров или использование BERT как экстрактора признаков с замороженными слоями.

Что важно проговорить на экзамене

- Transformer основан на механизме внимания, а не на рекуррентном проходе.
- Современные модели дают контекстуальные эмбеддинги.
- Предобучение использует большие неразмеченные корпуса.
- Transfer learning переносит языковые знания на прикладные задачи.
- Fine-tuning BERT обычно добавляет небольшую голову и дообучает модель под задачу.

24 Билет 24. Машинный перевод

Короткий план ответа

1. Перечислить основные технологии машинного перевода.
2. Описать правиловый, статистический и нейросетевой перевод.
3. Раскрыть принципы построения SMT и NMT.
4. Назвать ограничения технологий машинного перевода.

Короткая версия для письменного ответа

Машинный перевод — автоматическое преобразование текста с одного языка на другой с сохранением смысла. Основные подходы: перевод на правилах, статистический перевод, нейросетевой перевод и гибридные системы.

Перевод на правилах

RBMT использует лингвистические правила, словари и грамматики.

Варианты:

- прямой перевод: слово в слово с локальными перестановками;
- трансферный перевод: анализ исходного текста, преобразование структуры, генерация текста на целевом языке;
- интерлингва: перевод через универсальное семантическое представление.

Ограничения: огромная трудоемкость правил, хрупкость, слабая адаптация к новым доменам, трудности с идиомами и неформальным языком.

Статистический машинный перевод

SMT обучается на параллельных корпусах и ищет наиболее вероятный перевод.

По Байесу:

$$e^* = \arg\max_e P(e \mid f) = \arg\max_e P(f \mid e)P(e),$$

где $P(f \mid e)$ — модель перевода, а $P(e)$ — языковая модель целевого языка.

Основные варианты:

- word-based SMT;
- phrase-based SMT;
- syntax-based SMT.

Phrase-based SMT строится через выравнивание слов, извлечение пар фраз, оценку вероятностей фраз, модель переупорядочивания, языковую модель и декодер. Декодирование часто использует лучевой поиск.

Ограничения SMT: зависимость от больших параллельных корпусов, слабое моделирование дальних зависимостей, сложность настройки множества компонентов.

Нейросетевой машинный перевод

NMT напрямую моделирует $P(e \mid f)$ нейронной сетью, обычно в архитектуре encoder-decoder.

Развитие архитектур:

- RNN/LSTM/GRU encoder-decoder;
- attention, позволяющий декодеру обращаться к разным частям исходного предложения;

- Transformer, современный стандарт на self-attention.

Построение NMT:

1. собрать параллельный корпус;
2. выполнить токенизацию и разбиение на подслова, например BPE или SentencePiece;
3. выбрать архитектуру и гиперпараметры;
4. обучить модель end-to-end по кросс-энтропии;
5. при переводе использовать авторегрессионную генерацию и beam search;
6. оценивать качество метриками BLEU, METEOR, TER и ручной оценкой.

Ограничения NMT: требовательность к данным и ресурсам, трудная интерпретация, ошибки на редких словах, падение качества на длинных предложениях, возможные галлюцинации и проблемы с терминологией.

Гибридные системы

Гибридные подходы комбинируют правила, статистические модели и нейросети: например, используют правила для предобработки терминологии или постредактирования результата NMT.

Что важно проговорить на экзамене

- RBMT основан на правилах и словарях.
- SMT выбирает перевод по вероятностной модели перевода и языковой модели.
- NMT обучает одну нейросетевую модель перевода end-to-end.
- Transformer-based NMT — современный стандарт.
- Все подходы ограничены данными, неоднозначностью, контекстом, терминологией и оценкой качества.

25 Билет 25. Извлечение информации и анализ тональности

Короткий план ответа

1. Определить извлечение информации из текстов.
2. Перечислить виды извлекаемых данных.
3. Сравнить подходы к извлечению информации.
4. Описать извлечение мнений и анализ тональности.

Короткая версия для письменного ответа

Извлечение информации — автоматическое преобразование неструктурированного текста в структурированные факты: сущности, отношения, события, шаблоны или мнения. В отличие от информационного поиска, IE извлекает не документы, а конкретные данные из текста.

Особенности задачи

- исходные тексты не имеют строгой структуры;
- заранее задаются типы нужной информации;
- важны лексическая вариативность, синтаксис, контекст и неоднозначность;
- системы часто зависят от предметной области;
- качество оценивается через precision, recall и F1.

Виды извлекаемых данных

Именованные сущности люди, организации, локации, даты, денежные суммы. Задача называется NER.

Отношения связи между сущностями, например works-for или located-in.

События тип события, триггер и аргументы с ролями.

Шаблоны заполнение заранее заданной структуры, например карточки происшествия.

Мнения объект, аспект, держатель мнения, полярность и интенсивность.

Подходы к IE

Правиловый подход использует регулярные выражения, газеттиры, лексико-синтаксические шаблоны и грамматики. Он интерпретируем и точен для узких задач, но плохо переносится на новые домены.

ML-подход обучается на размеченных корпусах. NER часто формулируется как разметка последовательности с BIO-метками. Используются HMM, CRF, BiLSTM-CRF, Transformer-модели. Relation Extraction часто решается как классификация пары сущностей в контексте.

Гибридные подходы совмещают правила и модели: например, газеттиры используются как признаки, а ML-модель фильтрует кандидатов.

Distant supervision автоматически создает шумную разметку из баз знаний: если две сущности связаны отношением в базе, предложения с этой парой считаются кандидатами для обучения.

OpenIE извлекает открытые тройки вида аргумент–отношение–аргумент без заранее фиксированного набора отношений.

Анализ тональности

Анализ тональности выявляет субъективную оценку в тексте: позитивную, негативную, нейтральную или более детальную шкалу.

Уровни:

- уровень документа;
- уровень предложения;
- аспектный уровень.

Аспектный анализ отвечает на вопросы: кто выражает мнение, о чем, по какому аспекту и с какой тональностью. Пример: "Камера отличная, но батарея слабая— камера позитивно, батарея негативно.

Подходы:

- словари тональности: подсчет позитивных и негативных слов с учетом отрицаний и усилителей;
- машинное обучение: BoW, n-граммы, эмбеддинги, Naive Bayes, логистическая регрессия, SVM, CNN/RNN/Transformer;
- нейросетевые модели для аспектов, контекста, сарказма и сложных формулировок.

Сложности: сарказм, ирония, доменная зависимость лексики, отрицания, смешанные оценки и неявные мнения.

Что важно проговорить на экзамене

- IE извлекает структурированные факты из неструктурированного текста.
- Основные виды: сущности, отношения, события, шаблоны и мнения.
- NER часто решается как sequence labeling.
- Анализ тональности может быть документным, предложенческим или аспектным.
- Правила интерпретируемы, ML лучше масштабируется при наличии данных.

26 Билет 26. Процессная модель управления проектом

Короткий план ответа

1. Дать определение управления проектом и процессной модели.
2. Перечислить группы процессов управления проектом.
3. Описать взаимодействие процессов.
4. Назвать особенности управления программным проектом.

Короткая версия для письменного ответа

Управление проектом — применение знаний, навыков, инструментов и методов к проектной деятельности для достижения целей проекта. **Процессная модель** рассматривает управление проектом как набор взаимосвязанных процессов с входами, инструментами, методами и выходами.

Группы процессов

Инициация формально запускает проект или фазу. Результаты: устав проекта, назначение менеджера, определение заинтересованных сторон.

Планирование уточняет цели и способы достижения результата. Включает сбор требований, определение содержания, WBS, расписание, бюджет, качество, ресурсы, коммуникации, риски, закупки и план работы со стейкхолдерами.

Исполнение выполняет работы по плану: создание результатов, управление командой, качеством, коммуникациями, знаниями, закупками и вовлечением заинтересованных сторон.

Мониторинг и контроль отслеживает фактическое исполнение, сравнивает его с планом, выявляет отклонения, управляет изменениями, контролирует содержание, сроки, стоимость, качество, риски и ресурсы.

Закрытие формально завершает проект или фазу: приемка результатов, архивирование, закрытие контрактов, фиксация извлеченных уроков.

Взаимодействие процессов

Группы процессов не являются строго последовательными фазами. Планирование, исполнение и контроль часто идут параллельно и повторяются итеративно. Контроль порождает запросы на изменения, которые могут возвращать проект к перепланированию.

Типовая логика:

1. инициация дает основание проекта;
2. планирование задает базовые планы;
3. исполнение создает продукт;
4. мониторинг сравнивает факт с планом;
5. изменения проходят контроль и обновляют планы;
6. закрытие фиксирует результат и опыт.

Особенности программного проекта

Управление разработкой ПО имеет специфику:

- требования часто меняются по мере понимания задачи;
- трудоемкость и сроки трудно оценивать точно;
- результат нематериален и сложен для визуальной проверки;

- качество требует тестирования на разных уровнях;
- велико влияние квалификации и коммуникации команды;
- важны управление версиями, конфигурациями и зависимостями;
- возможны технический долг и архитектурные риски;
- часто применяются итеративные и гибкие подходы: Scrum, Kanban, инкрементальная разработка.

Программный продукт включает программы, процедуры, правила, документацию и данные, поставляемые пользователю.

Процесс разработки ПО обычно включает требования, проектирование, кодирование, тестирование, развертывание, сопровождение и обновления.

Что важно проговорить на экзамене

- Процессная модель описывает управление проектом через группы процессов.
- Основные группы: инициация, планирование, исполнение, мониторинг и контроль, закрытие.
- Процессы взаимодействуют итеративно, особенно через управление изменениями.
- В программных проектах особенно важны требования, качество, команда, версии и изменения.
- Agile-подходы полезны при высокой неопределенности и изменчивости требований.

27 Билет 27. Сетевое планирование проекта

Короткий план ответа

1. Дать определение сетевого планирования.
2. Описать работы, события, зависимости и сетевые графики.
3. Раскрыть CPM, PERT, критический путь и расчет сроков.
4. Объяснить назначение и выравнивание ресурсов.

Короткая версия для письменного ответа

Сетевое планирование — метод планирования и контроля проекта через граф работ и зависимостей. Он помогает определить длительность проекта, критический путь, резервы времени и влияние ограниченных ресурсов.

Основные понятия

Работа задача проекта, требующая времени и ресурсов.
Событие момент начала или окончания работы, не имеющий длительности.
Сетевой график граф работ и логических зависимостей.
Зависимость ограничение порядка выполнения работ.
Путь последовательность связанных работ от начала к завершению.
Критический путь самый длинный путь проекта; его длительность задает минимальный срок проекта.
Резерв времени допустимая задержка работы без сдвига срока проекта.

Типы зависимостей: Finish-to-Start, Start-to-Start, Finish-to-Finish, Start-to-Finish. Наиболее распространена Finish-to-Start.

Методы графиков

- **AOA/ADM**: работы изображаются дугами, события — узлами. Иногда нужны фиктивные работы нулевой длительности.
- **AON/PDM**: работы изображаются узлами, зависимости — стрелками. Этот способ чаще используется в современном ПО управления проектами.

AOA явно показывает события, но сложнее при изменениях и требует фиктивных работ. AON обычно нагляднее и проще для сложных проектов.

CPM и PERT

CPM — метод критического пути. Использует детерминированные длительности работ и вычисляет ранние/поздние сроки и резервы.

PERT применяется при неопределенности длительностей и использует три оценки: оптимистическую, наиболее вероятную и пессимистическую.

Расчет временных параметров

Для каждой работы:

- ES — раннее начало;
- $EF = ES + Dur$ — раннее окончание;
- LF — позднее окончание;
- $LS = LF - Dur$ — позднее начало;

- $TF = LS - ES = LF - EF$ — полный резерв.

Расчет:

1. **Прямой проход**: от начала к концу, считаем ES и EF. Для работы ES равен максимуму EF предшественников.
2. **Обратный проход**: от конца к началу, считаем LF и LS. Для работы LF равен минимуму LS последователей.
3. Работы с $TF = 0$ образуют критический путь.

Любая задержка критической работы задерживает весь проект, если не принять компенсирующие меры.

Диаграмма Ганта

Диаграмма Ганта показывает работы как полосы на временной шкале. Она удобна для визуализации календарного расписания, параллельных работ, длительностей и фактического прогресса. Сетевой график лучше показывает логику зависимостей, а диаграмма Ганта — календарное представление.

Назначение ресурсов

После построения графика назначаются ресурсы: люди, оборудование, материалы, финансы.

Процесс:

1. определить ресурсы для каждой работы;
2. оценить доступность ресурсов;
3. назначить ресурсы на работы;
4. построить загрузку ресурсов;
5. выявить перегрузки и недогрузки;
6. выполнить выравнивание или сглаживание ресурсов.

Resource leveling может сдвигать работы и даже увеличивать длительность проекта, если ресурсов не хватает. **Resource smoothing** оптимизирует загрузку в пределах уже заданных сроков, используя резервы некритических работ.

Способы выравнивания: сдвиг некритических работ в пределах резерва, разделение задач, привлечение дополнительных ресурсов, изменение длительности или состава работ.

Что важно проговорить на экзамене

- Сетевое планирование показывает зависимости между работами.
- Критический путь определяет минимальную длительность проекта.
- Резерв времени показывает, насколько работу можно задержать без сдвига проекта.
- CPM использует прямой и обратный проходы для расчета сроков.
- Назначение ресурсов может изменить реальное расписание даже при неизменной логике зависимостей.